

18-661 Introduction to Machine Learning

Support Vector Machines (SVM) – I

Spring 2026

ECE – Carnegie Mellon University

Announcements and Reminders

Recitation cancelled this Friday, 2/13.

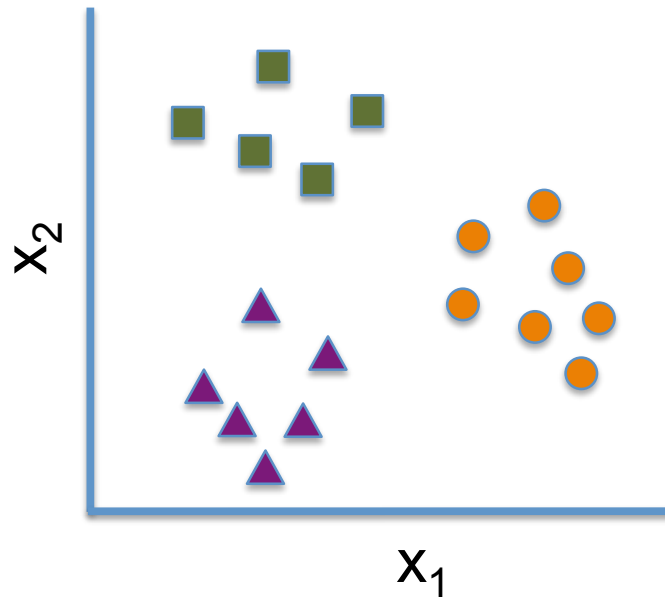
- Homework 2 is posted and due February 20.
- Midterm exam in two weeks, on February 25. More details to be announced in Wednesday's lecture next week.
 - Similar format to the mini-exam, but longer (110 minutes).
 - Covers all topics through the February 18 lecture.
from MLE/MAP
linear algebra

1. Review of Multi-class Logistic Regression
2. Support Vector Machines (SVM): Intuition
3. SVM: Max-Margin Formulation
4. SVM: Hinge Loss Formulation

Review of Multi-class Logistic Regression

Three Approaches

- One-versus-all
- One-versus-one
- Multinomial regression

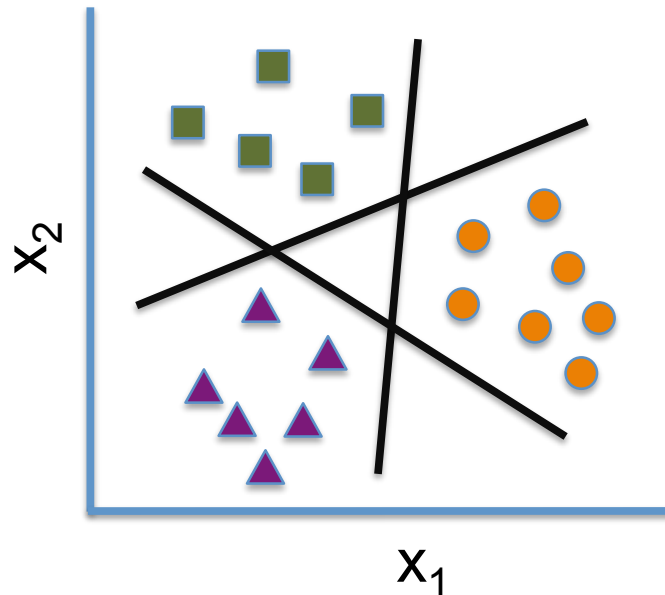


The One-versus-Rest or One-versus-All Approach

- For each class c , change the problem into binary classification
 1. Relabel training data with label c , into POSITIVE (or '1').
 2. Relabel all the rest data into NEGATIVE (or '0').
- Repeat this multiple times: Train C binary classifiers, using logistic regression to differentiate the two classes each time.
- Declare class c^* that maximizes

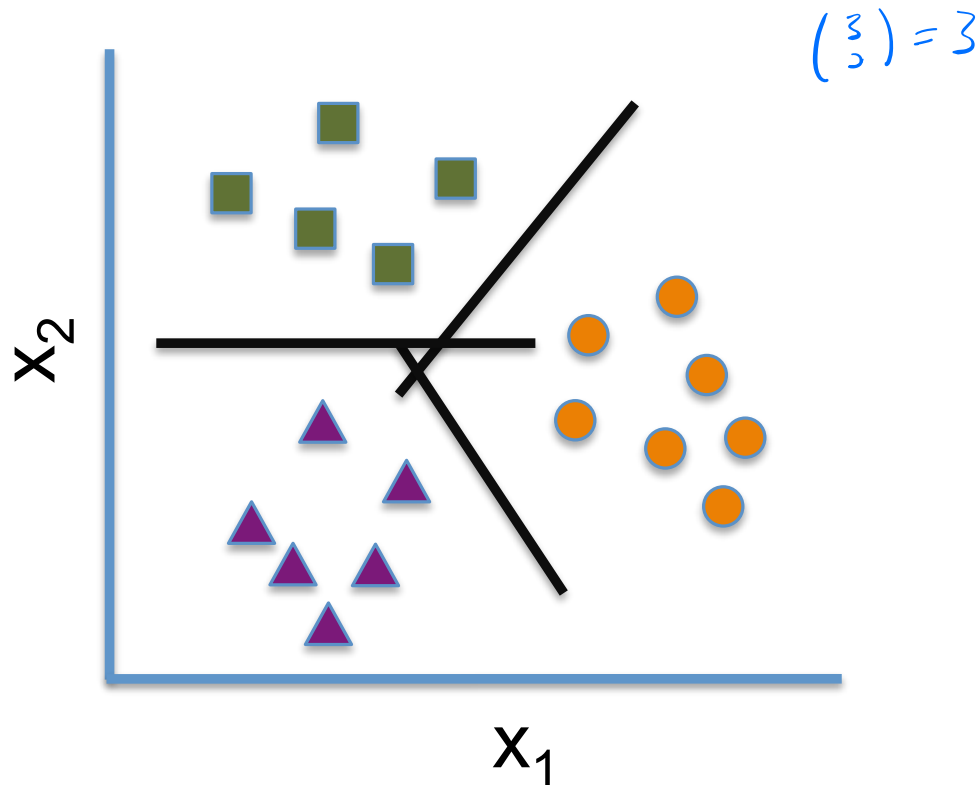
given input $\mathbf{x}$

$$c^* = \arg \max_{c=1,\dots,C} \Pr(y = c | \mathbf{x}) = \sigma(\mathbf{w}_c^\top \mathbf{x})$$



The One-versus-One Approach

- For each **pair** of classes c and c' , change the problem into binary classification.
 1. Relabel training data with label c , into POSITIVE (or '1')
 2. Relabel training data with label c' into NEGATIVE (or '0')
 3. **Disregard** all other data $\binom{C}{2}$
- Given $\mathbf{x}$, count the $\frac{C(C-1)}{2}$ votes from outputs of all binary classifiers and declare the winner as the predicted class.



Contrast These Approaches

Number of binary classifiers to be trained

- One-versus-All: C classifiers.
- One-versus-One: $C(C - 1)/2$ classifiers – bad if C is large
 $\binom{C}{2}$

Contrast These Approaches

Number of binary classifiers to be trained

- **One-versus-All:** C classifiers.
- **One-versus-One:** $C(C - 1)/2$ classifiers – bad if C is large

Effect of relabeling and splitting training data

- **One-versus-All:** imbalance in the number of positive and negative samples can cause bias in each trained classifier.
- **One-versus-One:** each classifier trained on a small subset of data (only data in two classes), which can result in high variance.

Contrast These Approaches

Number of binary classifiers to be trained

- **One-versus-All:** C classifiers.
- **One-versus-One:** $C(C - 1)/2$ classifiers – bad if C is large

Effect of relabeling and splitting training data

- **One-versus-All:** imbalance in the number of positive and negative samples can cause bias in each trained classifier.
- **One-versus-One:** each classifier trained on a small subset of data (only data in two classes), which can result in high variance.

Any other ideas?

Contrast These Approaches

Number of binary classifiers to be trained

- **One-versus-All:** C classifiers.
- **One-versus-One:** $C(C - 1)/2$ classifiers – bad if C is large

Effect of relabeling and splitting training data

- **One-versus-All:** imbalance in the number of positive and negative samples can cause bias in each trained classifier.
- **One-versus-One:** each classifier trained on a small subset of data (only data in two classes), which can result in high variance.

Any other ideas?

- **Hierarchical classification** – we will see this in decision trees

Contrast These Approaches

Number of binary classifiers to be trained

- **One-versus-All:** C classifiers.
- **One-versus-One:** $C(C - 1)/2$ classifiers – bad if C is large

Effect of relabeling and splitting training data

- **One-versus-All:** imbalance in the number of positive and negative samples can cause bias in each trained classifier.
- **One-versus-One:** each classifier trained on a small subset of data (only data in two classes), which can result in high variance.

Any other ideas?

- **Hierarchical classification** – we will see this in decision trees
- **Multinomial logistic regression** – directly output probabilities of y being in each of the C classes.

Multinomial Logistic Regression

Intuition:

from the decision rule of our Naïve Bayes classifier

$$\begin{aligned} y^* &= \arg \max_c P(y = c | \mathbf{x}) = \arg \max_c \log p(\mathbf{x} | y = c) p(y = c) \\ &= \arg \max_c \log \pi_c + \sum_k x_k \log \theta_{ck} = \arg \max_c \underbrace{\mathbf{w}_c^\top \mathbf{x}}_{\substack{\uparrow \\ \text{log conditional} \\ \text{probabilities}}} \quad \leftarrow \text{input data} \end{aligned}$$

Multinomial Logistic Regression

Intuition:

from the decision rule of our Naïve Bayes classifier

$$\begin{aligned} y^* &= \arg \max_c P(y = c | \mathbf{x}) = \arg \max_c \log p(\mathbf{x} | y = c) p(y = c) \\ &= \arg \max_c \log \pi_c + \sum_k x_k \log \theta_{ck} = \arg \max_c \mathbf{w}_c^\top \mathbf{x} \end{aligned}$$

Essentially, we are comparing

$$\mathbf{w}_1^\top \mathbf{x}, \mathbf{w}_2^\top \mathbf{x}, \dots, \mathbf{w}_C^\top \mathbf{x}$$

with **one** for each category.

So, can we define the following conditional model?

$$P(y = c | \mathbf{x}) = \sigma[\mathbf{w}_c^\top \mathbf{x}].$$


parameters
for class c

So, can we define the following conditional model?

$$P(y = c|\mathbf{x}) = \sigma[\mathbf{w}_c^\top \mathbf{x}].$$

This would **not** work because:

$$\sum_c P(y = c|\mathbf{x}) = \sum_c \sigma[\mathbf{w}_c^\top \mathbf{x}] \neq 1, \quad \text{should be } = 1$$

so each summand can be any number (independently) between 0 and 1.

So, can we define the following conditional model?

$$P(y = c|\mathbf{x}) = \sigma[\mathbf{w}_c^\top \mathbf{x}].$$

This would **not** work because:

$$\sum_c P(y = c|\mathbf{x}) = \sum_c \sigma[\mathbf{w}_c^\top \mathbf{x}] \neq 1,$$

so each summand can be any number (independently) between 0 and 1.

But we are close!

Learn the C linear models jointly to ensure this property holds!

Multinomial Logistic Regression

- **Model:** For each class c , we have a parameter vector $\mathbf{w}_c$ and model the posterior probability as:

$$\underline{P(c|\mathbf{x})} = \frac{e^{\mathbf{w}_c^\top \mathbf{x}}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}}} \quad \leftarrow \quad \text{This is called the } \textit{softmax} \text{ function.}$$

$$\in [0, 1]$$

$$\sum_{c'} P(c'|\mathbf{x}) = 1 \quad \underline{\text{valid}} \text{ probability estimate}$$

Multinomial Logistic Regression

- **Model:** For each class c , we have a parameter vector $\mathbf{w}_c$ and model the posterior probability as:

$$P(c|\mathbf{x}) = \frac{e^{\mathbf{w}_c^\top \mathbf{x}}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}}} \quad \leftarrow \quad \text{This is called the } \textit{softmax} \text{ function.}$$

- **Decision boundary:** Assign $\mathbf{x}$ with the label that is the maximum of posterior:

$$\arg \max_c P(c|\mathbf{x}) \rightarrow \arg \max_c \mathbf{w}_c^\top \mathbf{x}.$$

Handwritten notes and diagram:

$\arg \max_c$ is written in red. A red box highlights the numerator $e^{\mathbf{w}_c^\top \mathbf{x}}$ in the softmax formula. A red arrow points from this box to the $\arg \max_c$ term in the simplified equation above. Below the box, the denominator $\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}}$ is written in blue. To the left of the denominator, the text "does not change w/ class c " is written in red, with a red arrow pointing to the denominator.

How Does the Softmax Function Behave?

Suppose we have *3 classes*

$$\mathbf{w}_1^\top \mathbf{x} = 100, \quad \mathbf{w}_2^\top \mathbf{x} = 50, \quad \mathbf{w}_3^\top \mathbf{x} = -20.$$

We would pick the *winning* class label 1.

How Does the Softmax Function Behave?

Suppose we have

$$\mathbf{w}_1^\top \mathbf{x} = 100, \quad \mathbf{w}_2^\top \mathbf{x} = 50, \quad \mathbf{w}_3^\top \mathbf{x} = -20.$$

We would pick the **winning** class label 1.

Softmax translates these scores into well-formed conditional probabilities

$$P(y = 1|\mathbf{x}) = \frac{e^{100}}{e^{100} + e^{50} + e^{-20}} < 1$$

How Does the Softmax Function Behave?

Suppose we have

$$\mathbf{w}_1^\top \mathbf{x} = 100, \quad \mathbf{w}_2^\top \mathbf{x} = 50, \quad \mathbf{w}_3^\top \mathbf{x} = -20.$$

We would pick the **winning** class label 1.

Softmax translates these scores into well-formed conditional probabilities

$$P(y = 1|\mathbf{x}) = \frac{e^{100}}{e^{100} + e^{50} + e^{-20}} < 1$$

- Preserves relative ordering of scores.
- Maps scores to values between 0 and 1 that also sum to 1.

Parameter Estimation for Multinomial Logistic Regression

Discriminative approach: Maximize conditional likelihood

$$\log P(\mathcal{D}) = \sum_n \log P(y_n | \mathbf{x}_n)$$

↗
data
points

Parameter Estimation for Multinomial Logistic Regression

Discriminative approach: Maximize conditional likelihood

$$\log P(\mathcal{D}) = \sum_n \log P(y_n | \mathbf{x}_n)$$

Handwritten notes: $0, 1, 2, \dots, C$ with an arrow pointing to y_n in the equation.

We will change y_n to $\mathbf{y}_n = [y_{n1} \ y_{n2} \ \cdots \ y_{nC}]^\top$, a C -dimensional vector using 1-of- C encoding.

Handwritten note: one-hot encoding

$$y_{nc} = \begin{cases} 1 & \text{if } y_n = c \\ 0 & \text{otherwise} \end{cases}$$

Ex: if $y_n = 2$, then, $\mathbf{y}_n = [0 \ \mathbf{1} \ 0 \ 0 \ \cdots \ 0]^\top$.

Handwritten notes: "class 2" under $y_n = 2$; "class 1" with an arrow pointing to the 0 before the 1; "class 2" with an arrow pointing to the 1.

Parameter Estimation for Multinomial Logistic Regression

Discriminative approach: Maximize conditional likelihood

$$\log P(\mathcal{D}) = \sum_n \log P(y_n | \mathbf{x}_n)$$

We will change y_n to $\mathbf{y}_n = [y_{n1} \ y_{n2} \ \cdots \ y_{nC}]^\top$, a C -dimensional vector using 1-of- C encoding.

$$y_{nc} = \begin{cases} 1 & \text{if } y_n = c \\ 0 & \text{otherwise} \end{cases}$$

Ex: if $y_n = 2$, then, $\mathbf{y}_n = [0 \ \mathbf{1} \ 0 \ 0 \ \cdots \ 0]^\top$.

$$\Rightarrow \sum_n \log P(y_n | \mathbf{x}_n) = \sum_n \log \prod_{c=1}^C P(c | \mathbf{x}_n)^{y_{nc}} = \sum_n \sum_c y_{nc} \log P(c | \mathbf{x}_n)$$

Handwritten annotations:

- A blue cloud-like shape around the first part of the equation contains the text: $\prod_{c=1}^C P(c | \mathbf{x}_n)^{y_{nc}}$ with a double vertical line above it.
- A blue arrow points from the handwritten $0, 1$ to the y_{nc} term in the product.
- A blue arrow points from the handwritten "from 1-hot encoding" to the y_{nc} term in the sum.
- A blue arrow points from the handwritten "data points" to the n index in the sum.
- A blue arrow points from the handwritten "classes" to the c index in the sum.
- A blue arrow points from the handwritten "softmax" to the $P(c | \mathbf{x}_n)$ term.

Cross-entropy Error Function

Definition: negative log-likelihood

$$\mathcal{E}(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_C) = - \sum_n \sum_c y_{nc} \log P(c|\mathbf{x}_n)$$

softmax

$$= - \sum_n \sum_c y_{nc} \log \left(\frac{e^{\mathbf{w}_c^\top \mathbf{x}_n}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}_n}} \right)$$

depends on $\mathbf{w}_c$

minimize

Cross-entropy Error Function

Definition: negative log-likelihood

$$\begin{aligned}\mathcal{E}(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_C) &= - \sum_n \sum_c y_{nc} \log P(c|\mathbf{x}_n) \\ &= - \sum_n \sum_c y_{nc} \log \left(\frac{e^{\mathbf{w}_c^\top \mathbf{x}_n}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}_n}} \right)\end{aligned}$$

Properties of cross-entropy

- Convex in the $\mathbf{w}$ vectors, therefore local minimum = global optimum

Cross-entropy Error Function

Definition: negative log-likelihood

$$\begin{aligned}\mathcal{E}(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_C) &= - \sum_n \sum_c y_{nc} \log P(c|\mathbf{x}_n) \\ &= - \sum_n \sum_c y_{nc} \log \left(\frac{e^{\mathbf{w}_c^\top \mathbf{x}_n}}{\sum_{c'} e^{\mathbf{w}_{c'}^\top \mathbf{x}_n}} \right)\end{aligned}$$

Properties of cross-entropy

- Convex in the $\mathbf{w}$ vectors, therefore local minimum = global optimum
- Optimization requires numerical procedures, analogous to those used for binary logistic regression.

1. Review of Multi-class Logistic Regression
2. Support Vector Machines (SVM): Intuition
3. SVM: Max-Margin Formulation
4. SVM: Hinge Loss Formulation

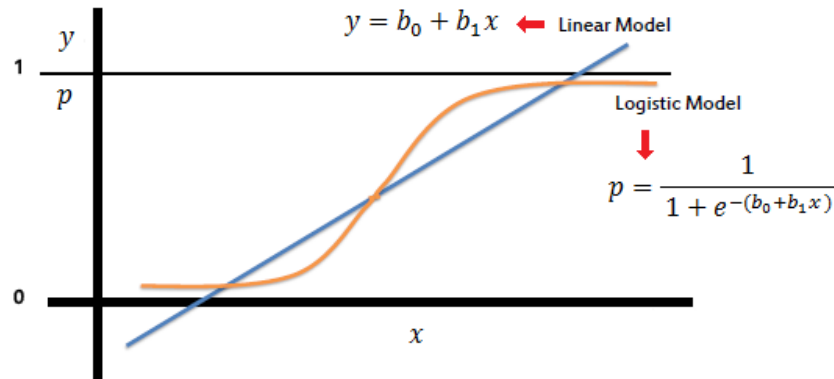
Support Vector Machines (SVM): Intuition

Why Do We Need SVM?

Alternative to Logistic Regression and Naïve Bayes.

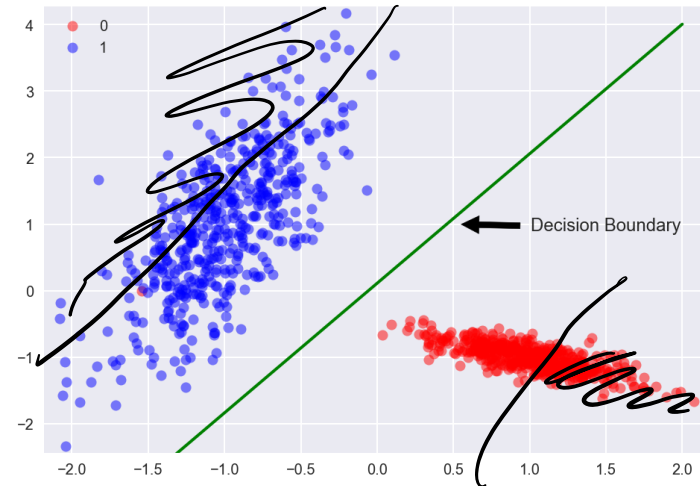
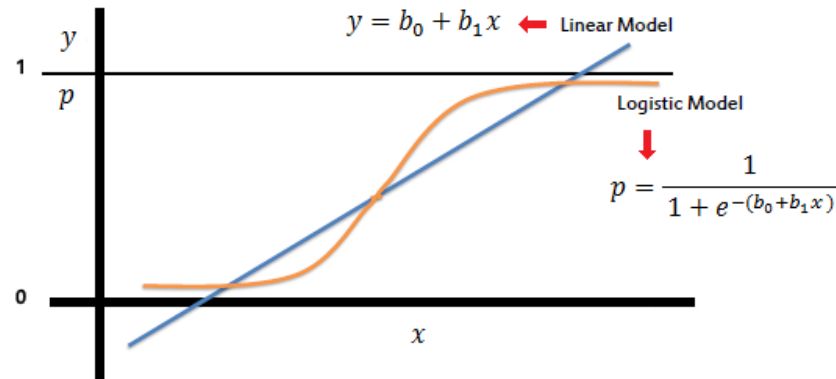
- Logistic regression and Naïve Bayes train over the whole dataset.
- These can require a lot of memory in high-dimensional settings.
- SVM can give a better and more efficient solution.
- SVM is one of the most powerful and commonly used ML algorithms (besides neural networks).

Binary Logistic Regression



- We only need to know if $p(\mathbf{x}) > 0.5$ or < 0.5 .
- We **don't** (always) need to know how far $\mathbf{x}$ is from this boundary.

Binary Logistic Regression



- We only need to know if $p(\mathbf{x}) > 0.5$ or < 0.5 .
- We **don't** (always) need to know how far $\mathbf{x}$ is from this boundary.

How can we use this insight to improve the classification algorithm?

- What if we just looked at the boundary?
- Maybe then we could ignore some of the samples?

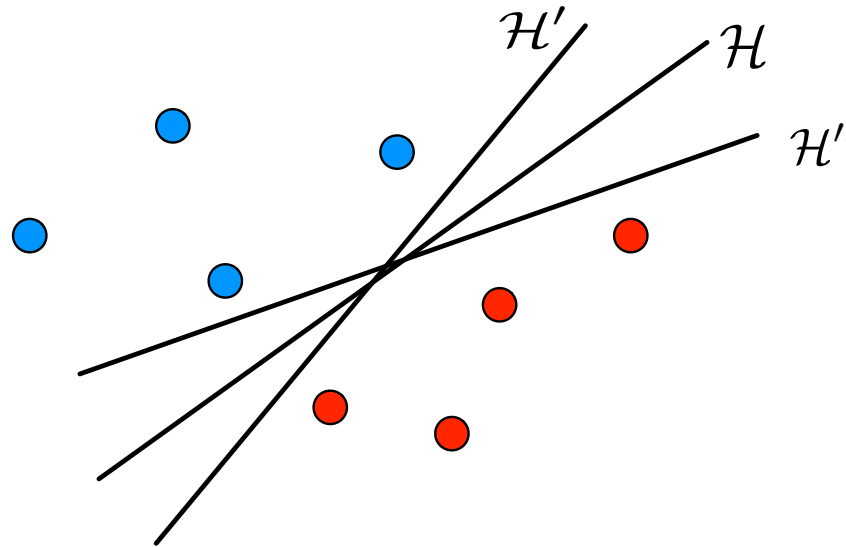
Advantages of SVM

We will see later that SVM:

1. Maximizes distance of training points from the boundary
2. Only requires a subset of the training points.
3. Is less sensitive to outliers.
4. Scales better with high-dimensional data.
5. Generalizes well to many nonlinear models.

SVM: Max-Margin Formulation

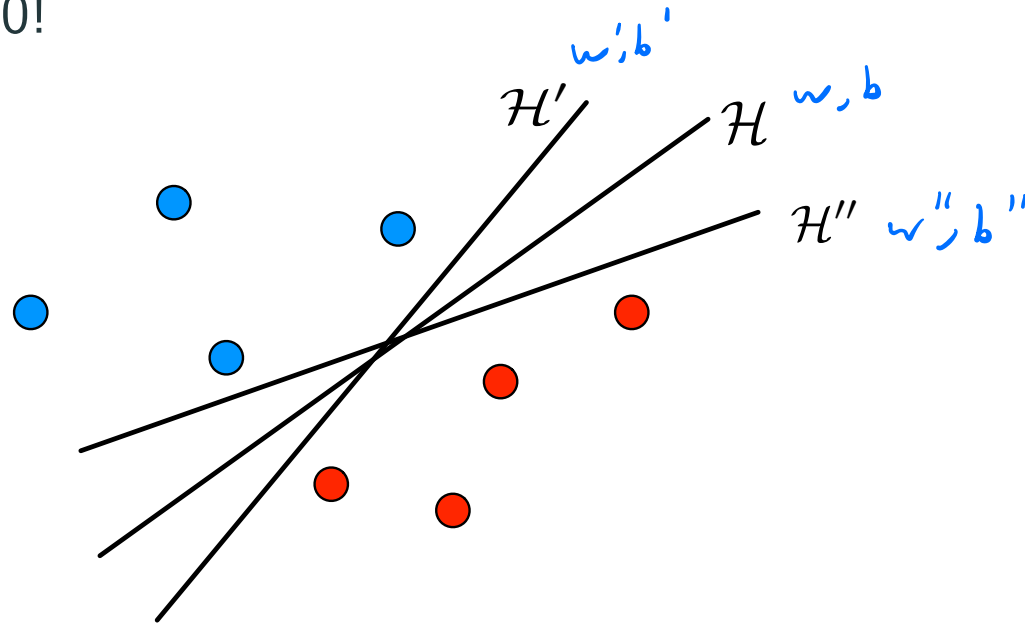
Binary Classification: Finding a Linear Decision Boundary



- Input features $\mathbf{x}$.
- Decision boundary is a hyperplane $\mathcal{H} : \underbrace{\mathbf{w}^\top \mathbf{x} + b}_{w, b} = 0$.

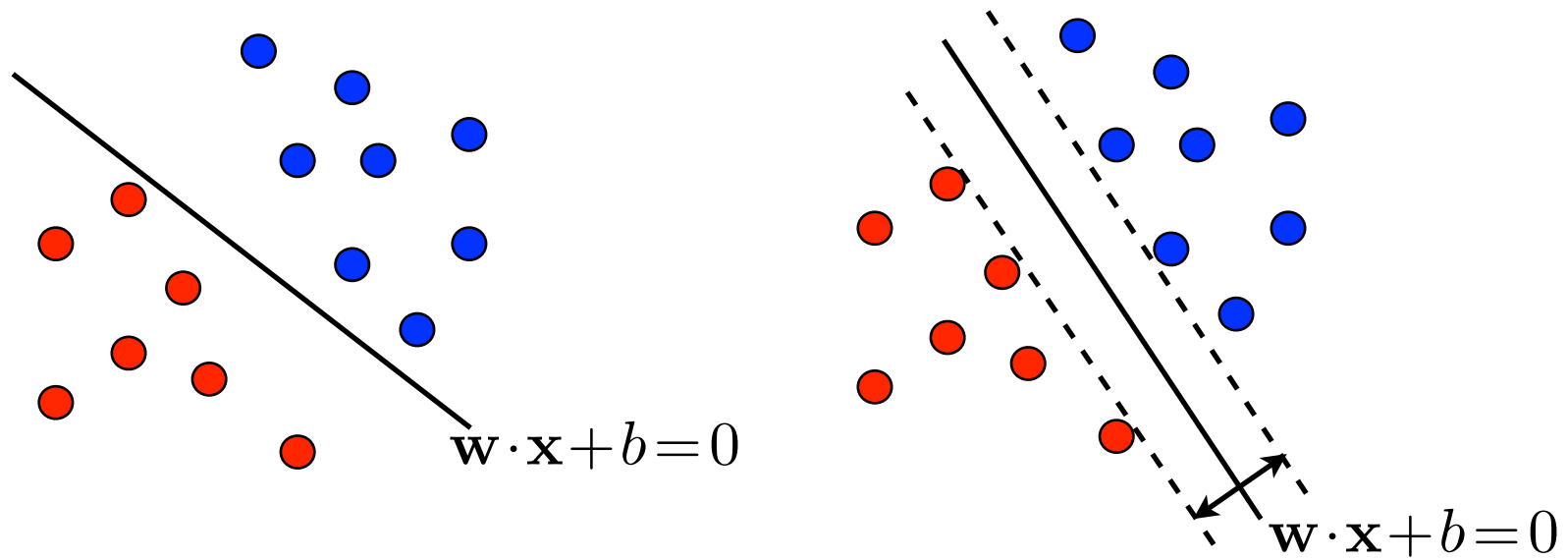
Intuition: Where to Put the Decision Boundary?

- Consider a *separable* training dataset (e.g., with two features)
- There are an **infinite** number of decision boundaries
 $\mathcal{H} : \mathbf{w}^\top \mathbf{x} + b = 0!$

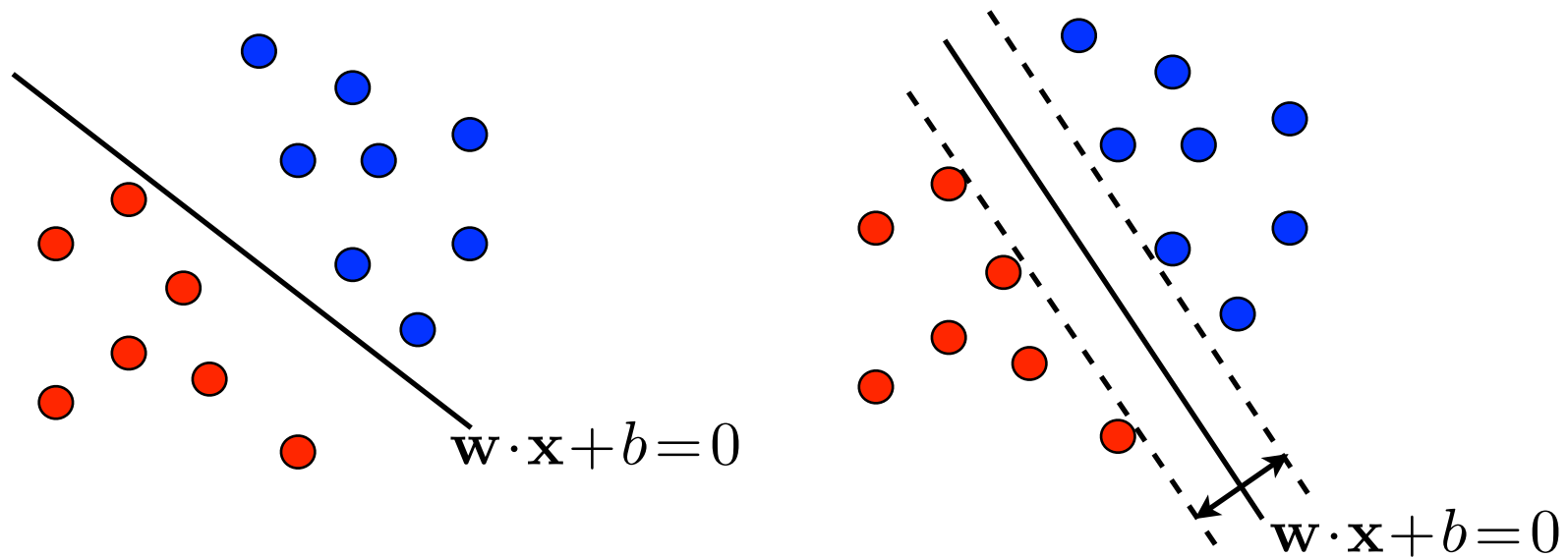


- Which one should we pick?

Intuition: Where to Put the Decision Boundary?



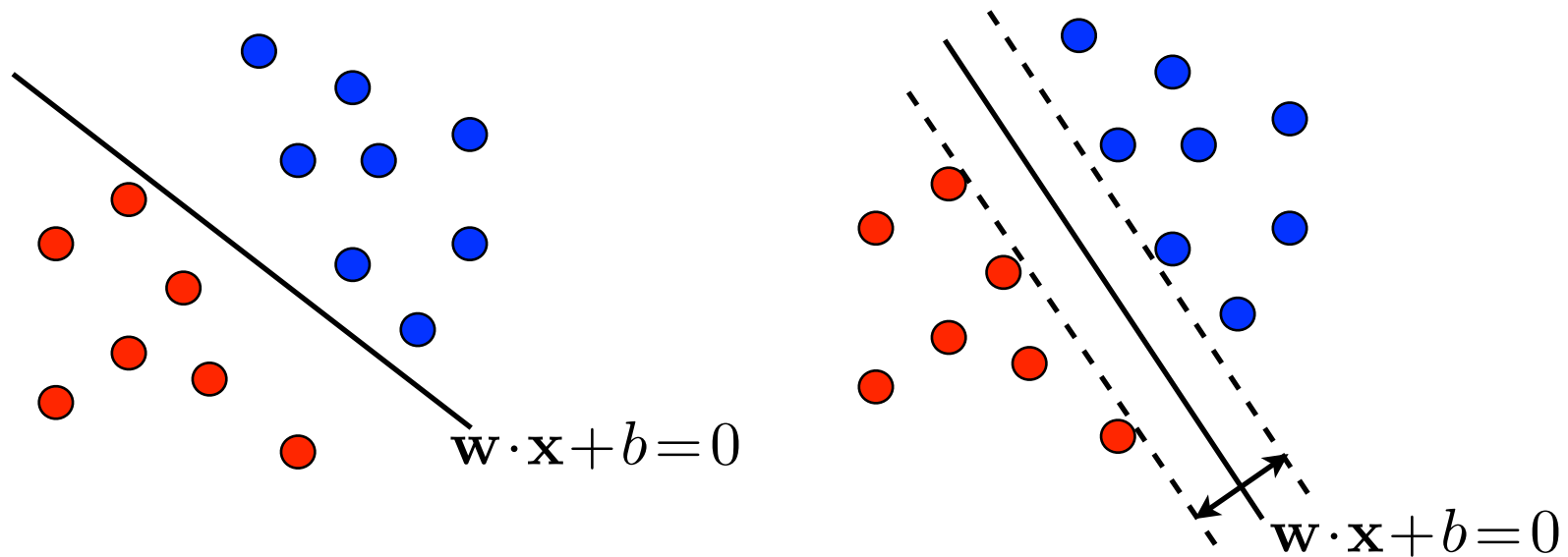
Intuition: Where to Put the Decision Boundary?



Find a decision boundary in the 'middle' of the two classes that:

- Perfectly classifies the training data
- Is as far away from every training point as possible

Intuition: Where to Put the Decision Boundary?



Find a decision boundary in the '*middle*' of the two classes that:

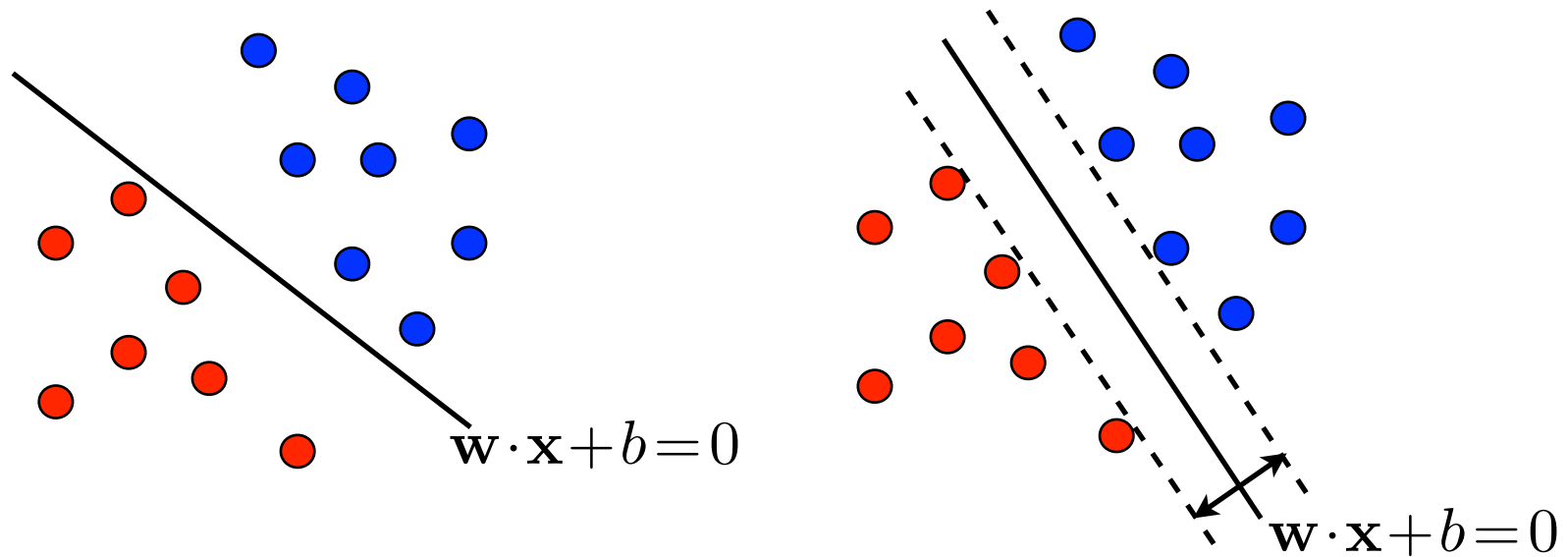
- Perfectly classifies the training data
- Is as far away from every training point as possible

distance
data to
boundary

Let us apply this intuition to build a classifier that maximizes the margin between training points and the decision boundary.

First, Some Vector Geometry

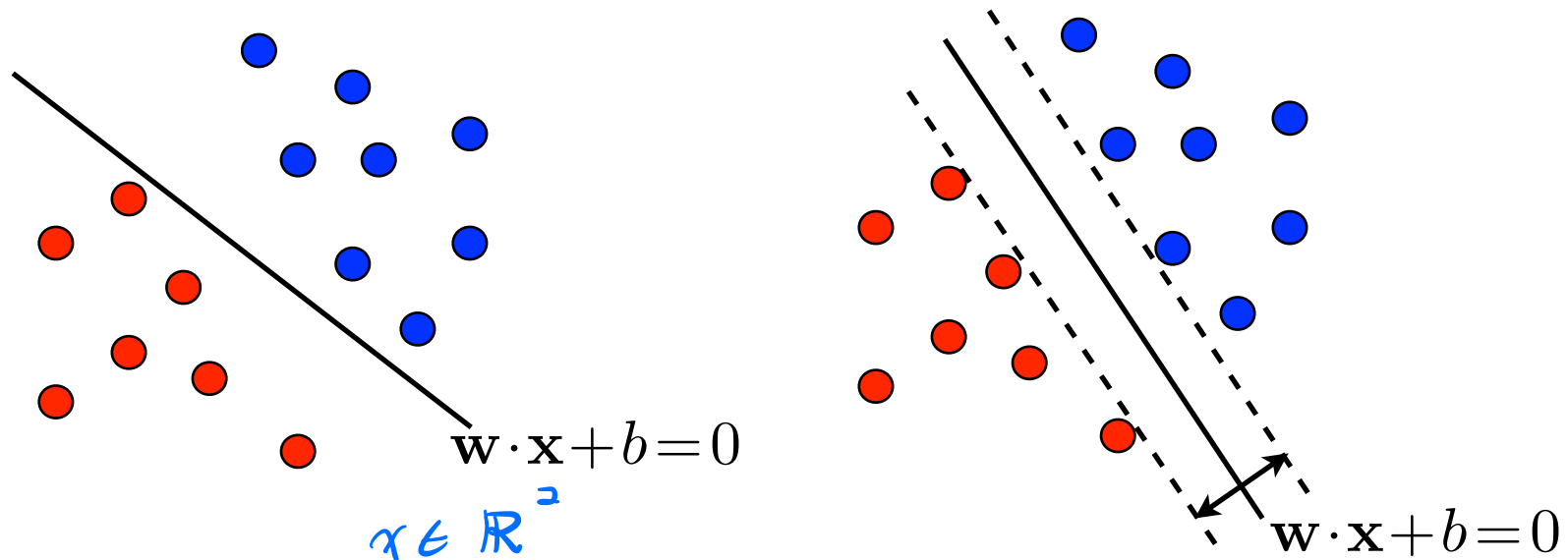
What is a hyperplane?



- General equation is $\mathbf{w}^\top \mathbf{x} + b = 0$
- Divides the space in half, i.e., $\mathbf{w}^\top \mathbf{x} + b > 0$ and $\mathbf{w}^\top \mathbf{x} + b < 0$

First, Some Vector Geometry

What is a hyperplane?



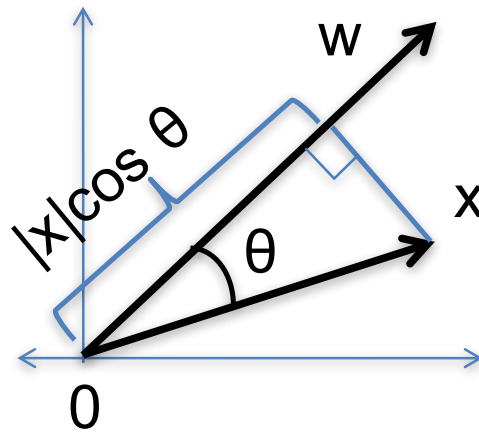
- General equation is $\mathbf{w}^\top \mathbf{x} + b = 0$
- Divides the space in half, i.e., $\mathbf{w}^\top \mathbf{x} + b > 0$ and $\mathbf{w}^\top \mathbf{x} + b < 0$
- A hyperplane is a line in 2D and a plane in 3D
- $\mathbf{w} \in \mathbb{R}^d$ is a non-zero normal vector *as in pictures*

Vector Norms and Inner Products

- Given two vectors $\mathbf{w}$ and $\mathbf{x}$, what is their inner product?
- Inner Product $\mathbf{w}^\top \mathbf{x}$ $= w_1x_1 + w_2x_2 + \cdots + w_dx_d$

Vector Norms and Inner Products

- Given two vectors $\mathbf{w}$ and $\mathbf{x}$, what is their inner product?
- Inner Product $\mathbf{w}^\top \mathbf{x} = w_1x_1 + w_2x_2 + \cdots + w_dx_d$

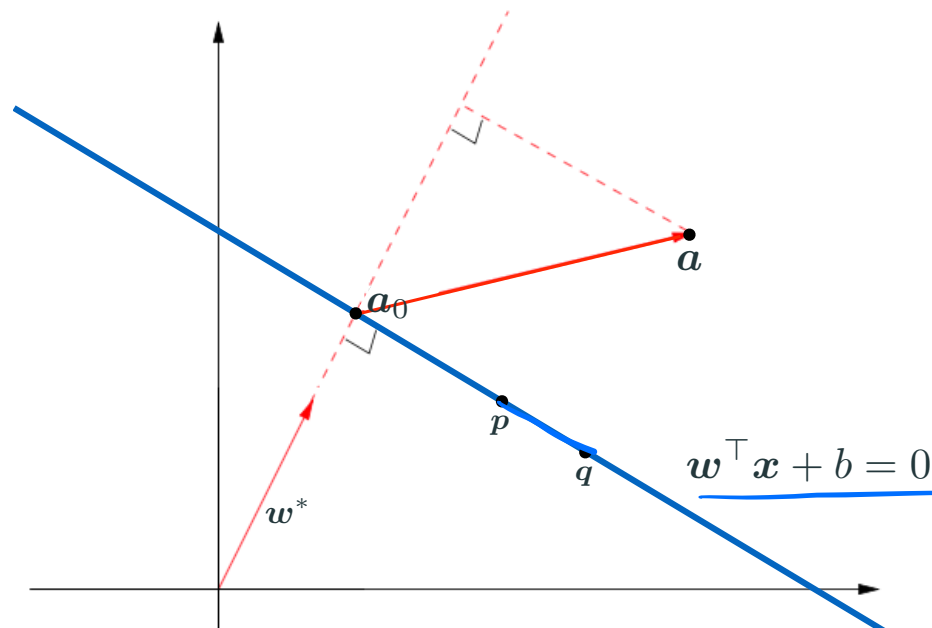


- Inner Product $\mathbf{w}^\top \mathbf{x}$ is also equal to $\|\mathbf{w}\| \|\mathbf{x}\| \cos \theta$
- $\mathbf{w}^\top \mathbf{w} = \|\mathbf{w}\|^2$ *← squared norm*
- If $\mathbf{w}$ and $\mathbf{x}$ are perpendicular, then $\theta = \pi/2$, and thus the inner product is zero.

l₂ norm

angle between w, x

Normal Vector of a Hyperplane

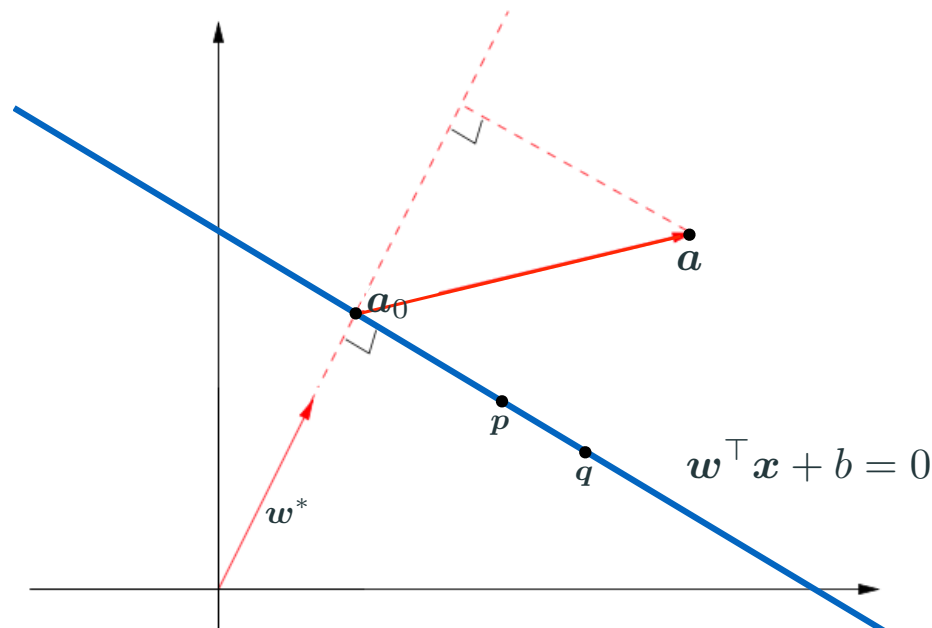


perpendicular

Vector w is normal to the hyperplane. Why?

- If p and q are both on the line, then $w^\top p + b = w^\top q + b = 0$.

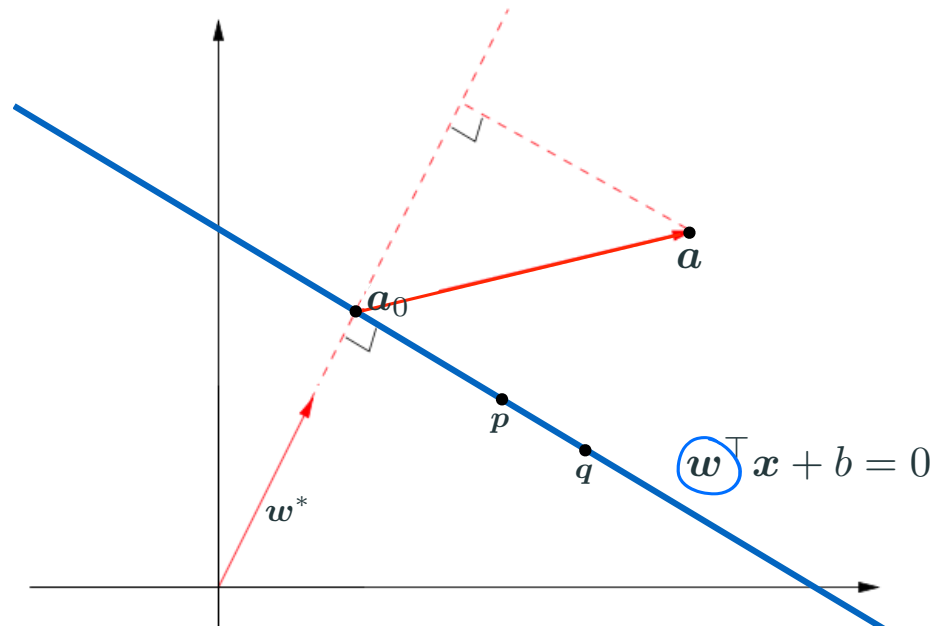
Normal Vector of a Hyperplane



Vector w is normal to the hyperplane. Why?

- If p and q are both on the line, then $w^\top p + b = w^\top q + b = 0$.
- Then $w^\top (p - q) = w^\top p - w^\top q = -b - (-b) = 0$
inner product $\Rightarrow w$ perpendicular to $p - q$

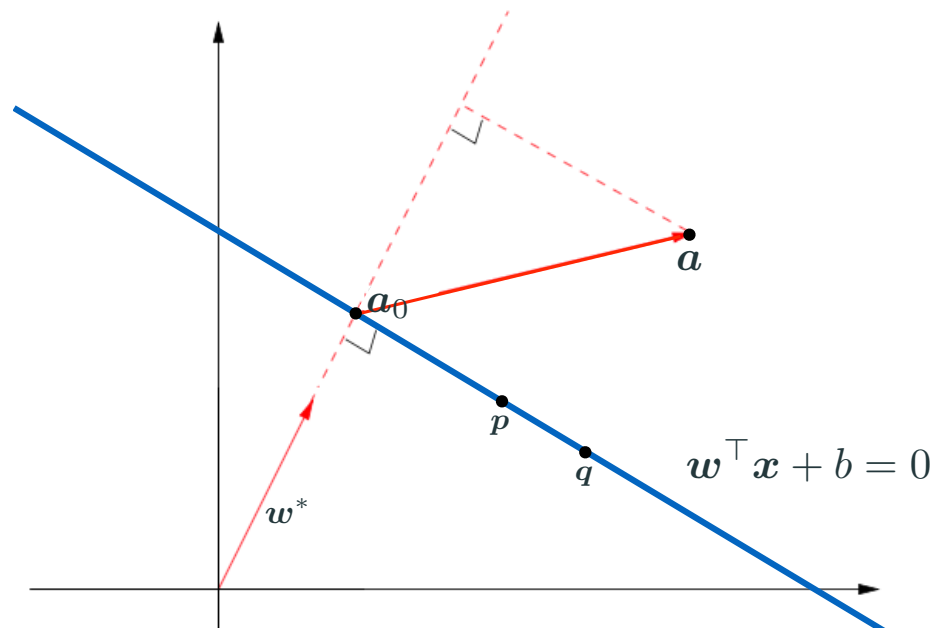
Normal Vector of a Hyperplane



Vector w is normal to the hyperplane. Why?

- If p and q are both on the line, then $w^T p + b = w^T q + b = 0$.
- Then $w^T (p - q) = w^T p - w^T q = -b - (-b) = 0$
- $p - q$ is an arbitrary vector parallel to the line, thus w is orthogonal

Normal Vector of a Hyperplane

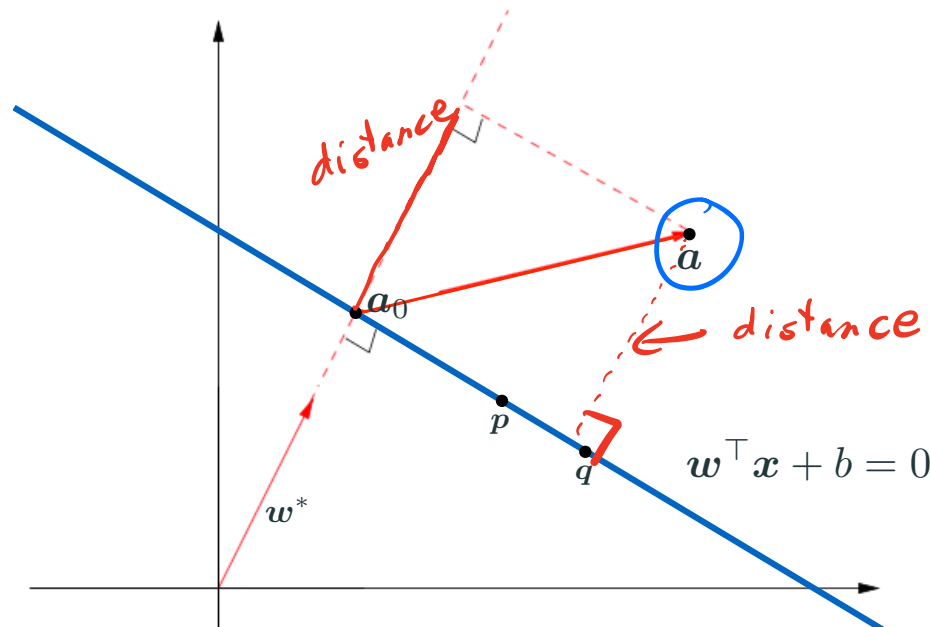


Vector w is normal to the hyperplane. Why?

- If p and q are both on the line, then $w^\top p + b = w^\top q + b = 0$.
- Then $w^\top (p - q) = w^\top p - w^\top q = -b - (-b) = 0$
- $p - q$ is an arbitrary vector parallel to the line, thus w is orthogonal
- $w^* = \frac{w}{\|w\|_2}$ is the unit normal vector

$$\|w^*\|_2 = 1 \quad \checkmark$$

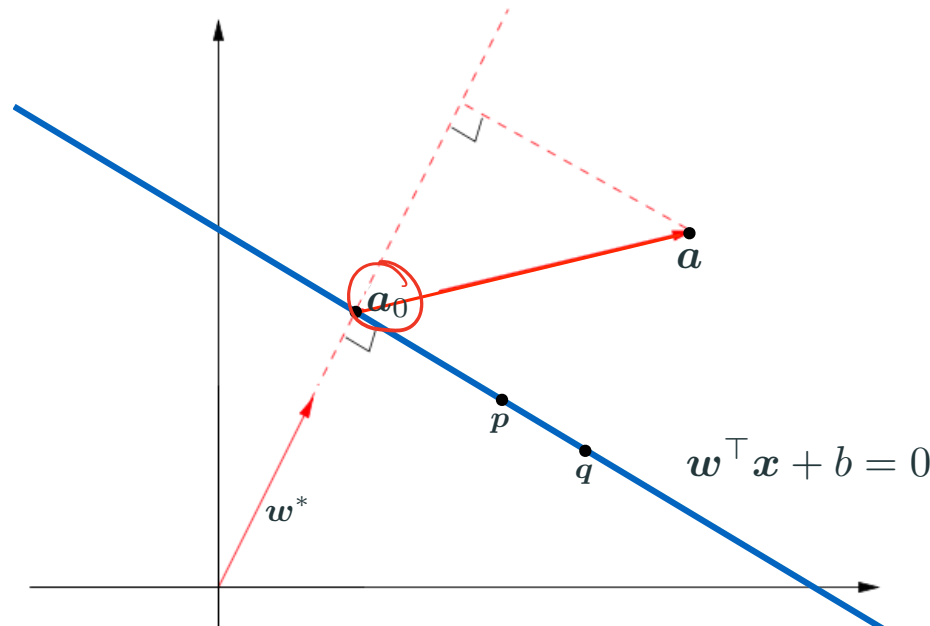
Distance from a Hyperplane



How to find the distance from a to the hyperplane?

- We want to find distance between a and line in the direction of w^* .
 $w^T x + b = 0$

Distance from a Hyperplane

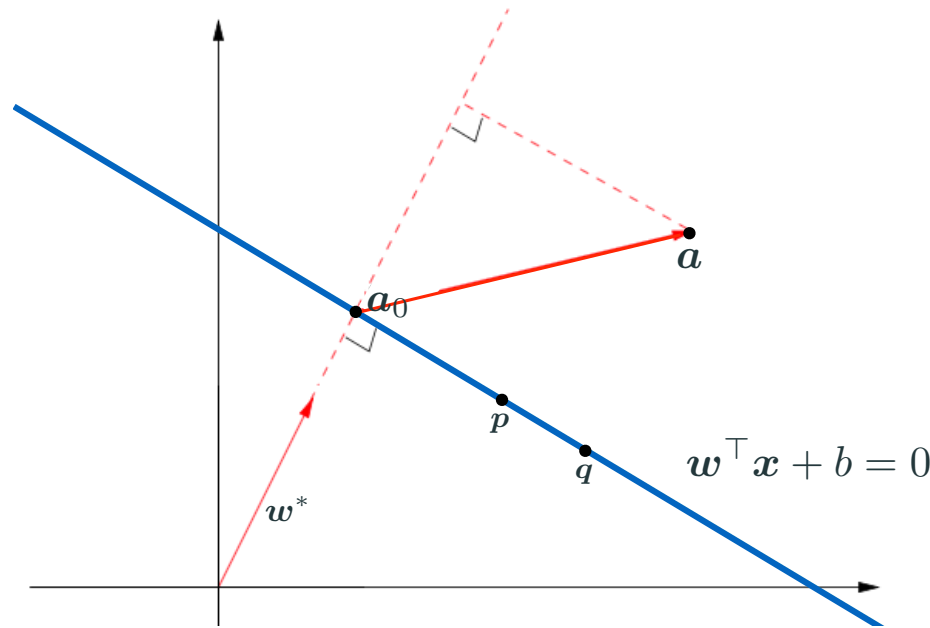


How to find the distance from a to the hyperplane?

- We want to find distance between a and line in the direction of w^* .
- If we define point a_0 on the line, then this distance corresponds to length of $a - a_0$ in direction of w^* , which equals $w^{*\top}(a - a_0)$.

projection

Distance from a Hyperplane

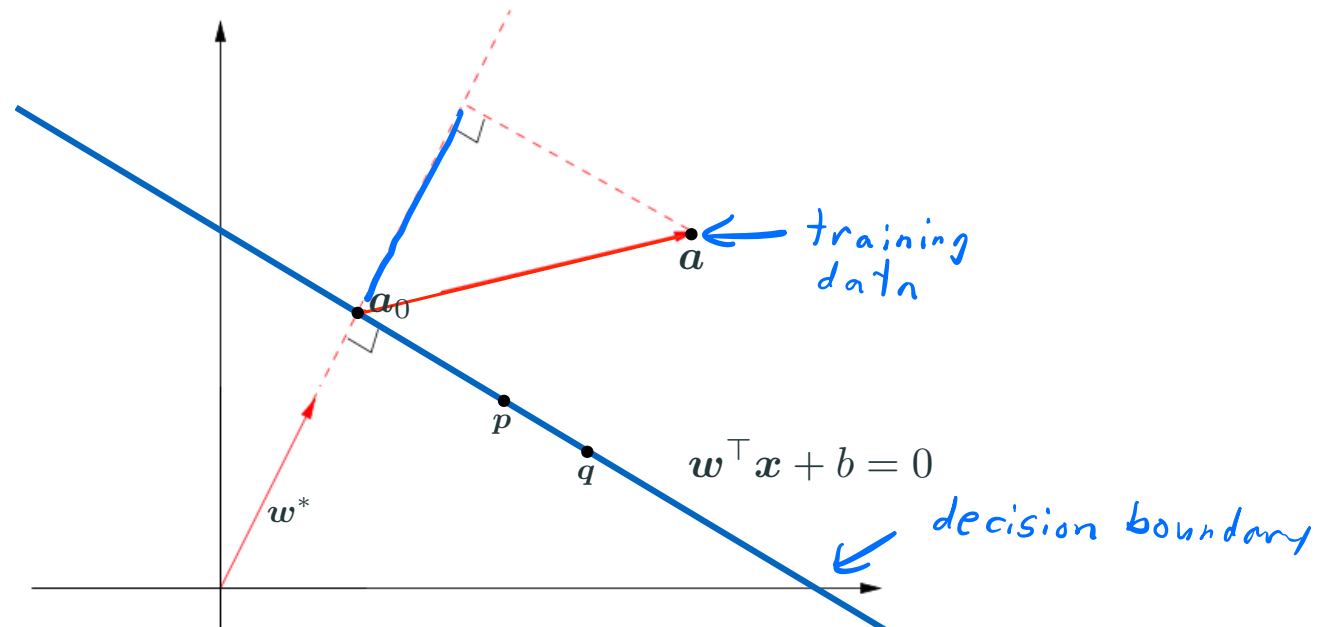


How to find the distance from $\mathbf{a}$ to the hyperplane?

- We want to find distance between $\mathbf{a}$ and line in the direction of $\mathbf{w}^*$.
- If we define point $\mathbf{a}_0$ on the line, then this distance corresponds to length of $\mathbf{a} - \mathbf{a}_0$ in direction of $\mathbf{w}^*$, which equals $\mathbf{w}^{*\top}(\mathbf{a} - \mathbf{a}_0)$.
- We know $\mathbf{w}^\top \mathbf{a}_0 = -b$ since $\mathbf{w}^\top \mathbf{a}_0 + b = 0$.

on the
hyperplane

Distance from a Hyperplane



How to find the distance from $\mathbf{a}$ to the hyperplane?

- We want to find distance between $\mathbf{a}$ and line in the direction of $\mathbf{w}^*$.
- If we define point $\mathbf{a}_0$ on the line, then this distance corresponds to length of $\mathbf{a} - \mathbf{a}_0$ in direction of $\mathbf{w}^*$, which equals $\mathbf{w}^{*\top}(\mathbf{a} - \mathbf{a}_0)$.
- We know $\mathbf{w}^\top \mathbf{a}_0 = -b$ since $\mathbf{w}^\top \mathbf{a}_0 + b = 0$.
- Then the distance equals $\frac{1}{\|\mathbf{w}\|_2} (\mathbf{w}^\top \mathbf{a} + b)$.

want to maximize for data points $\mathbf{a}$

Distance from a Point to Decision Boundary

The *unsigned* distance from a point $\overset{\text{training data}}{\downarrow} \mathbf{x}$ to the decision boundary (hyperplane) $\mathcal{H}$ is

$$d_{\mathcal{H}}(\mathbf{x}) = \frac{|\mathbf{w}^{\top} \mathbf{x} + b|}{\|\mathbf{w}\|_2}$$

Distance from a Point to Decision Boundary

The *unsigned* distance from a point $\mathbf{x}$ to the decision boundary (hyperplane) $\mathcal{H}$ is

$$d_{\mathcal{H}}(\mathbf{x}) = \frac{|\mathbf{w}^{\top} \mathbf{x} + b|}{\|\mathbf{w}\|_2}$$

How to remove the absolute value $|\cdot|$?

Distance from a Point to Decision Boundary

The *unsigned* distance from a point $\mathbf{x}$ to the decision boundary (hyperplane) $\mathcal{H}$ is

$$d_{\mathcal{H}}(\mathbf{x}) = \frac{|\mathbf{w}^{\top} \mathbf{x} + b|}{\|\mathbf{w}\|_2}$$

How to remove the absolute value $|\cdot|$? *(x_0) training data*

Notation changes from Logistic Regression: Use $y = +1$ to represent positive label and $y = -1$ for negative label.

Distance from a Point to Decision Boundary

The *unsigned* distance from a point $\mathbf{x}$ to the decision boundary (hyperplane) $\mathcal{H}$ is

$$d_{\mathcal{H}}(\mathbf{x}) = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|_2} \quad \leftarrow \text{vector geometry}$$

How to remove the absolute value $|\cdot|$?

Notation changes from Logistic Regression: Use $y = +1$ to represent positive label and $y = -1$ for negative label.

Then, exploiting the fact that the decision boundary classifies every point in the training dataset correctly, we have $(\mathbf{w}^T \mathbf{x} + b)$ and $\mathbf{x}$'s label y must have the same sign. So we get

if $y > 0, \mathbf{w}^T \mathbf{x} + b > 0$
 $y = 1$

if $y = -1, \mathbf{w}^T \mathbf{x} + b < 0$

$$|\mathbf{w}^T \mathbf{x} + b| = \frac{1}{y} (\mathbf{w}^T \mathbf{x} + b)$$

$$d_{\mathcal{H}}(\mathbf{x}) = \frac{y[\mathbf{w}^T \mathbf{x} + b]}{\|\mathbf{w}\|_2}$$

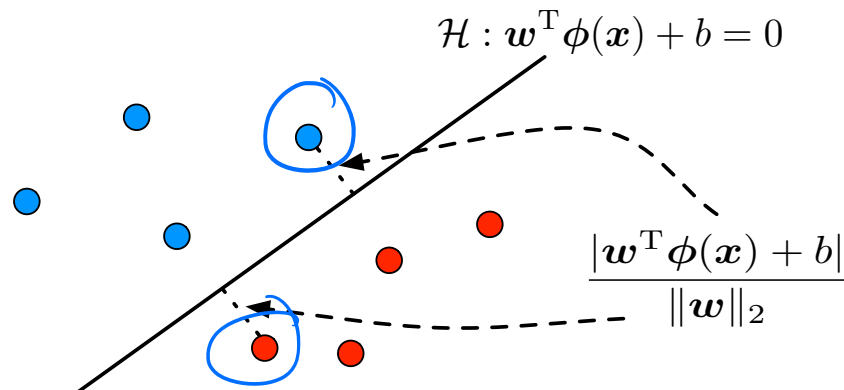
$|\mathbf{w}^T \mathbf{x} + b|$
training data pts.

Defining the Margin

Margin

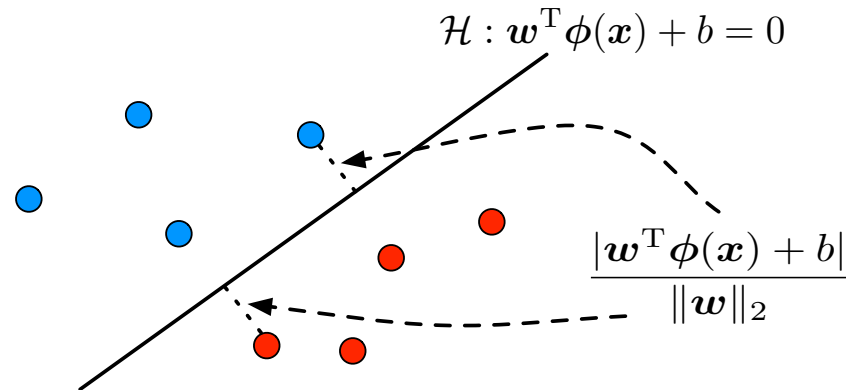
Smallest distance between the hyperplane and all training points

$$\text{MARGIN}(\mathbf{w}, b) = \min_n \frac{y_n [\mathbf{w}^\top \mathbf{x}_n + b]}{\|\mathbf{w}\|_2} \quad d_{\mathcal{H}}(\mathbf{x}_n)$$



How can we use this to find the SVM solution?

Optimizing the Margin



How should we pick $(\mathbf{w}, b)$ based on its margin?

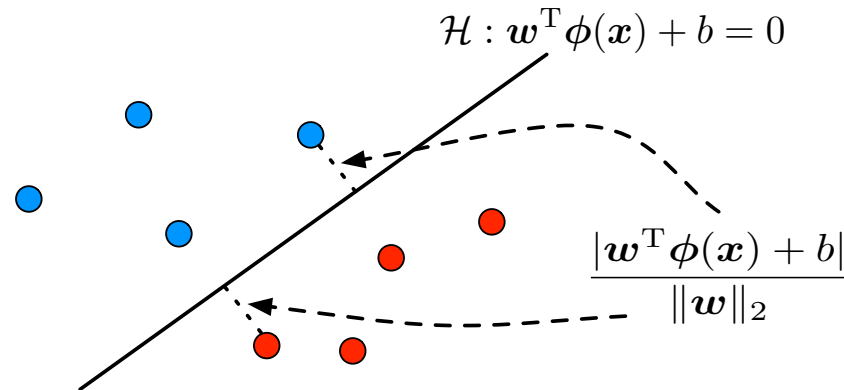
We want a decision boundary that is as far away from all training points as possible, so we to maximize the margin!

$$\max_{\mathbf{w}, b} \left(\min_n \frac{y_n [\mathbf{w}^T \mathbf{x}_n + b]}{\|\mathbf{w}\|_2} \right) = \max_{\mathbf{w}, b} \left(\frac{1}{\|\mathbf{w}\|_2} \min_n y_n [\mathbf{w}^T \mathbf{x}_n + b] \right)$$

↑
decision
boundary
(hyperplane)

margin for
a given $\mathbf{w}, b$

Optimizing the Margin



How should we pick $(\mathbf{w}, b)$ based on its margin?

We want a decision boundary that is as far away from all training points as possible, so we to *maximize* the margin!

$$\max_{\mathbf{w}, b} \left(\min_n \frac{y_n [\mathbf{w}^\top \mathbf{x}_n + b]}{\|\mathbf{w}\|_2} \right) = \max_{\mathbf{w}, b} \left(\frac{1}{\|\mathbf{w}\|_2} \min_n y_n [\mathbf{w}^\top \mathbf{x}_n + b] \right)$$

Only involves points near the boundary (more on this later).

Scale of w

Margin

Smallest distance between the hyperplane and all training points

$$\text{MARGIN}(\mathbf{w}, b) = \min_n \frac{y_n[\mathbf{w}^\top \mathbf{x}_n + b]}{\|\mathbf{w}\|_2} \rightarrow \min_n \frac{y_n[\cancel{2}\mathbf{w}^\top \mathbf{x}_n + \cancel{2}b]}{\cancel{2}\|\mathbf{w}\|_2}$$

Consider three hyperplanes

$$(\mathbf{w}, b) \quad (2\mathbf{w}, 2b) \quad (.5\mathbf{w}, .5b)$$

Which one has the largest margin? *they are the same*

Scale of w

Margin

Smallest distance between the hyperplane and all training points

$$\text{MARGIN}(\mathbf{w}, b) = \min_n \frac{y_n[\mathbf{w}^\top \mathbf{x}_n + b]}{\|\mathbf{w}\|_2}$$

Consider three hyperplanes

$$(\mathbf{w}, b) \quad (2\mathbf{w}, 2b) \quad (.5\mathbf{w}, .5b)$$

Which one has the largest margin?

- The MARGIN doesn't change if we scale $(\mathbf{w}, b)$ by a constant c
- $\mathbf{w}^\top \mathbf{x} + b = 0$ and $(c\mathbf{w})^\top \mathbf{x} + (cb) = 0$: same decision boundary!

Scale of w

Margin

Smallest distance between the hyperplane and all training points

$$\text{MARGIN}(\mathbf{w}, b) = \min_n \frac{y_n[\mathbf{w}^\top \mathbf{x}_n + b]}{\|\mathbf{w}\|_2}$$

Consider three hyperplanes

$$(\mathbf{w}, b) \quad (2\mathbf{w}, 2b) \quad (.5\mathbf{w}, .5b)$$

Which one has the largest margin?

- The MARGIN doesn't change if we scale $(\mathbf{w}, b)$ by a constant c
- $\mathbf{w}^\top \mathbf{x} + b = 0$ and $(c\mathbf{w})^\top \mathbf{x} + (cb) = 0$: same decision boundary!
- Can we further constrain the problem so as to get a unique solution $(\mathbf{w}, b)$?

Rescaled Margin

We can further constrain the problem by scaling $(\mathbf{w}, b)$ such that

$$\min_n y_n [\mathbf{w}^\top \mathbf{x}_n + b] = 1. \quad \leftarrow \text{contradicts}$$

Note that there always exists a scaling for which this is true.

$\Rightarrow \mathbf{w}, b$ perfectly separates training data

suppose misclassified $(\mathbf{x}, y)$

$$\text{sign}(y) \neq \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

$$y [\mathbf{w}^\top \mathbf{x} + b] < 0$$

Rescaled Margin

We can further constrain the problem by scaling $(\mathbf{w}, b)$ such that

$$\min_n y_n [\mathbf{w}^\top \mathbf{x}_n + b] = 1.$$

Note that there always exists a scaling for which this is true. We've fixed the numerator in the $\text{MARGIN}(\mathbf{w}, b)$ equation, and we have:

$$\text{MARGIN}(\mathbf{w}, b) = \frac{\min_n y_n [\mathbf{w}^\top \mathbf{x}_n + b]}{\|\mathbf{w}\|_2} = \frac{1}{\|\mathbf{w}\|_2}$$

Rescaled Margin

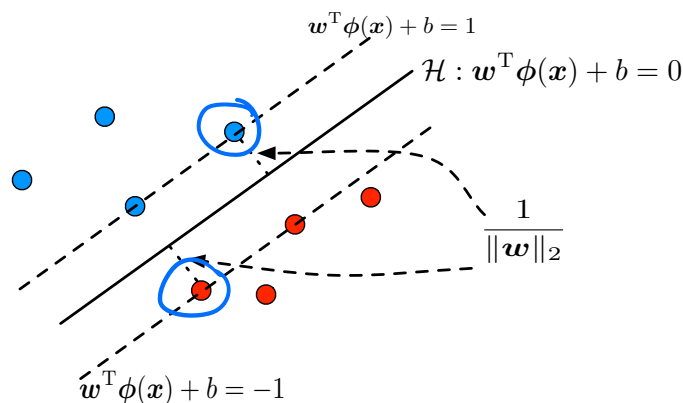
We can further constrain the problem by scaling $(\mathbf{w}, b)$ such that

$$\min_n y_n [\mathbf{w}^\top \mathbf{x}_n + b] = 1.$$

Note that there always exists a scaling for which this is true. We've fixed the numerator in the $\text{MARGIN}(\mathbf{w}, b)$ equation, and we have:

$$\text{MARGIN}(\mathbf{w}, b) = \frac{\min_n y_n [\mathbf{w}^\top \mathbf{x}_n + b]}{\|\mathbf{w}\|_2} = \frac{1}{\|\mathbf{w}\|_2}$$

Hence the points closest to the decision boundary are at distance $\frac{1}{\|\mathbf{w}\|_2}$.



SVM: Max-margin Formulation for Separable Data

We thus want to solve:

$$\max_{\mathbf{w}, b} \underbrace{\frac{1}{\|\mathbf{w}\|_2}}_{\text{margin} = \text{min distance training data : boundary}} \quad \text{such that} \quad \underbrace{\min_n y_n [\mathbf{w}^\top \mathbf{x}_n + b]}_{\text{scaling of } \mathbf{w}, b} = 1$$

which is equivalent to

$$\boxed{\max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|_2} \quad \text{such that} \quad y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n}$$

optimization problem

SVM: Max-margin Formulation for Separable Data

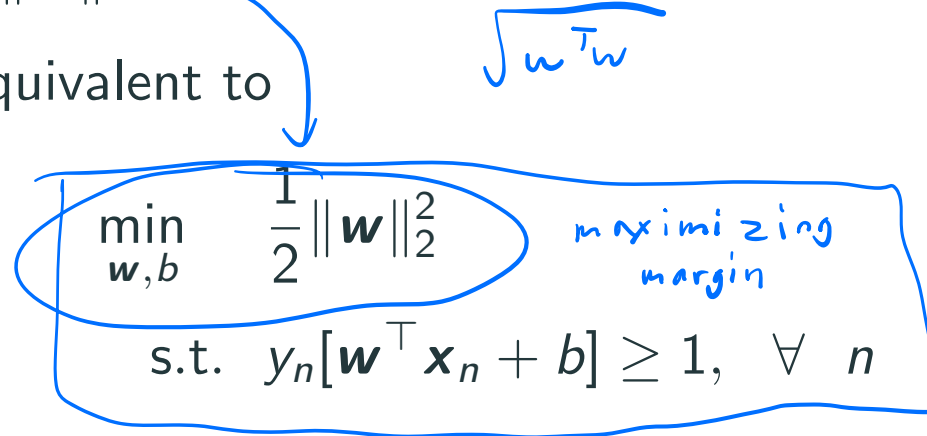
We thus want to solve:

$$\max_{\mathbf{w}, b} \underbrace{\frac{1}{\|\mathbf{w}\|_2}}_{\text{margin}} \quad \text{such that} \quad \underbrace{\min_n y_n [\mathbf{w}^\top \mathbf{x}_n + b]}_{\text{scaling of } \mathbf{w}, b} = 1$$

which is equivalent to

$$\max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|_2} \quad \text{such that} \quad y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n$$

This is further equivalent to

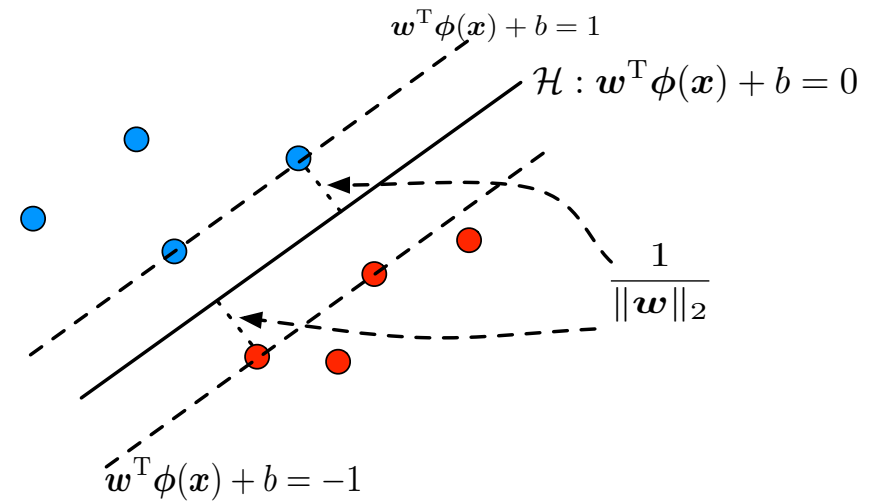

$$\begin{aligned} & \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 \quad \text{maximizing margin} \\ & \text{s.t. } y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n \end{aligned}$$

Given our geometric intuition, SVM is called a **max margin** (or large margin) classifier. The constraints are called **large margin constraints**.

Support Vectors: A First Look

SVM formulation for separable data

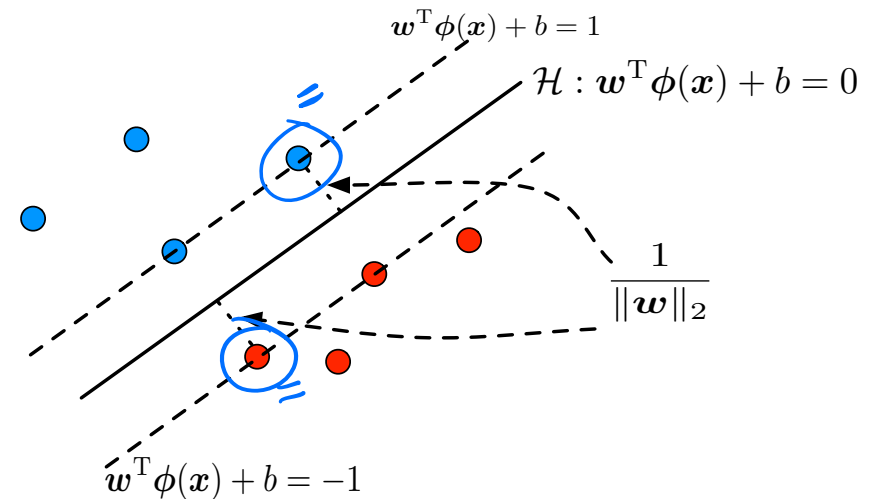
$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n \end{aligned}$$



Support Vectors: A First Look

SVM formulation for separable data

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & \underline{y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall \quad n} \\ & \text{all training data} \end{aligned}$$



Two types of training data, based on the situations of the constraint:

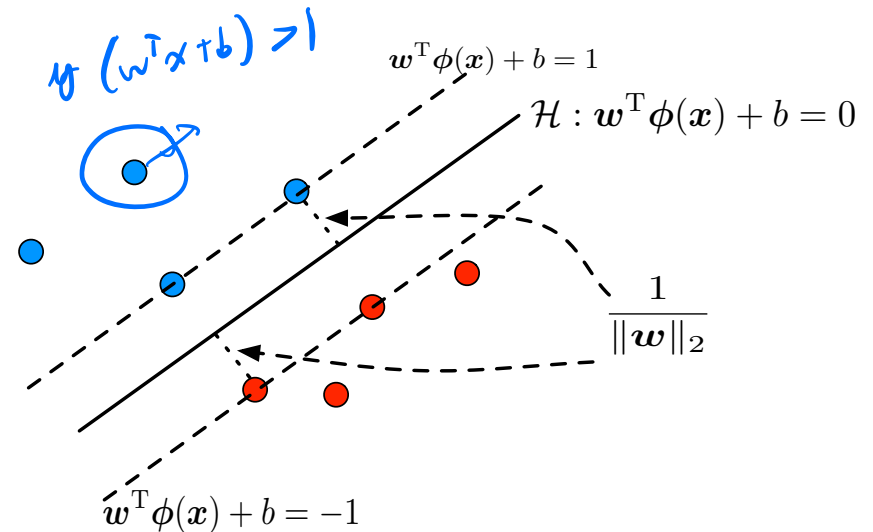
- “=”: $y_n[\mathbf{w}^\top \mathbf{x}_n + b] = 1$. These training data points are called “support vectors”, which have the minimum distance $(\frac{1}{\|\mathbf{w}\|})$ to the boundary.

Support Vectors: A First Look

SVM formulation for separable data

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|_2^2$$

$$\text{s.t. } y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n$$



Two types of training data, based on the situations of the constraint:

- “=”: $y_n[\mathbf{w}^\top \mathbf{x}_n + b] = 1$. These training data points are called “support vectors”, which have the minimum distance $(\frac{1}{\|\mathbf{w}\|})$ to the boundary.
- “>”: $y_n[\mathbf{w}^\top \mathbf{x}_n + b] > 1$. Distance to the boundary is larger than the minimum. Removing these data points does not affect the optimal solution (more on this next lecture).

SVM for Non-separable Data

SVM formulation for separable data

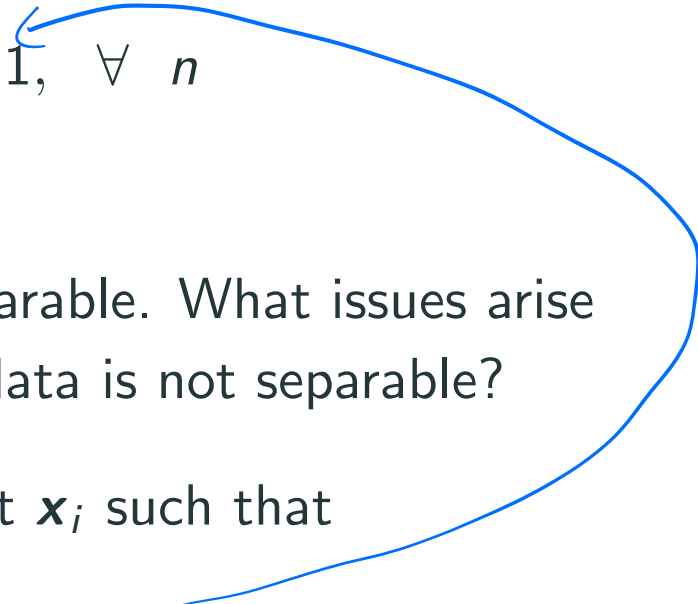
$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n \end{aligned}$$

Non-separable setting

In practice our training data may not be separable. What issues arise with the optimization problem above when data is not separable?

SVM for Non-separable Data

SVM formulation for separable data

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n \end{aligned}$$


Non-separable setting

In practice our training data may not be separable. What issues arise with the optimization problem above when data is not separable?

- For every $\mathbf{w}$ there exists a training point $\mathbf{x}_i$ such that

$$\begin{aligned} y_i[\mathbf{w}^\top \mathbf{x}_i + b] &\leq 0 \\ \text{sign}(y_i) &\neq \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b) \end{aligned}$$

SVM for Non-separable Data

SVM formulation for separable data

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n \end{aligned}$$

Non-separable setting

In practice our training data may not be separable. What issues arise with the optimization problem above when data is not separable?

- For every $\mathbf{w}$ there exists a training point $\mathbf{x}_i$ such that

$$y_i[\mathbf{w}^\top \mathbf{x}_i + b] \leq 0$$

- There is no feasible $(\mathbf{w}, b)$ as at least one of our constraints is violated!

SVM for Non-separable Data

Constraints in separable setting

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n$$

Constraints in non-separable setting

Can we modify our constraints to account for non-separability?

SVM for Non-separable Data

Constraints in separable setting

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n$$

Constraints in non-separable setting

Can we modify our constraints to account for non-separability?

Specifically, we introduce slack variables $\xi_n \geq 0$:

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq \underbrace{1 - \xi_n}_{\geq 1}, \quad \forall n$$

SVM for Non-separable Data

Constraints in separable setting

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n$$

Constraints in non-separable setting

Can we modify our constraints to account for non-separability?

Specifically, we introduce **slack variables** $\xi_n \geq 0$:

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n$$

- For “hard” training points, we can increase ξ_n until the above inequalities are met.

SVM for Non-separable Data

Constraints in separable setting

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n$$

Constraints in non-separable setting

Can we modify our constraints to account for non-separability?

Specifically, we introduce **slack variables** $\xi_n \geq 0$:

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n$$

- For “hard” training points, we can increase ξ_n until the above inequalities are met.
- What does it mean when ξ_n is very large?

SVM for Non-separable Data

Constraints in separable setting

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \quad \forall n$$

Constraints in non-separable setting

Can we modify our constraints to account for non-separability?

Specifically, we introduce **slack variables** $\xi_n \geq 0$:

$$y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n$$

- For “hard” training points, we can increase ξ_n until the above inequalities are met.
- What does it mean when ξ_n is very large? We have violated the original constraints “by a lot.”

Soft-margin SVM Formulation

We do not want ξ_n to grow too large, and we can control their size by incorporating them into our optimization problem:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

Soft-margin SVM Formulation

We do not want ξ_n to grow too large, and we can control their size by incorporating them into our optimization problem:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

What is the role of C ? *regularization constant*

Soft-margin SVM Formulation

We do not want ξ_n to grow too large, and we can control their size by incorporating them into our optimization problem:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

What is the role of C ?

- User-defined hyperparameter

Soft-margin SVM Formulation

We do not want ξ_n to grow too large, and we can control their size by incorporating them into our optimization problem:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

What is the role of C ?

- User-defined hyperparameter
- Trades off between the two terms in our objective

Soft-margin SVM Formulation

We do not want ξ_n to grow too large, and we can control their size by incorporating them into our optimization problem:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

Handwritten notes:
A blue arrow points from the ξ_n term in the constraint to the text $y_n (\mathbf{w}^\top \mathbf{x}_n + b) = 0$ decision boundary.
Below the constraint, $\xi_n \Rightarrow 0$ is written in blue.

What is the role of C ?

- User-defined hyperparameter
- Trades off between the two terms in our objective
- Same idea as the regularization term in ridge regression

How to Solve this Problem?

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

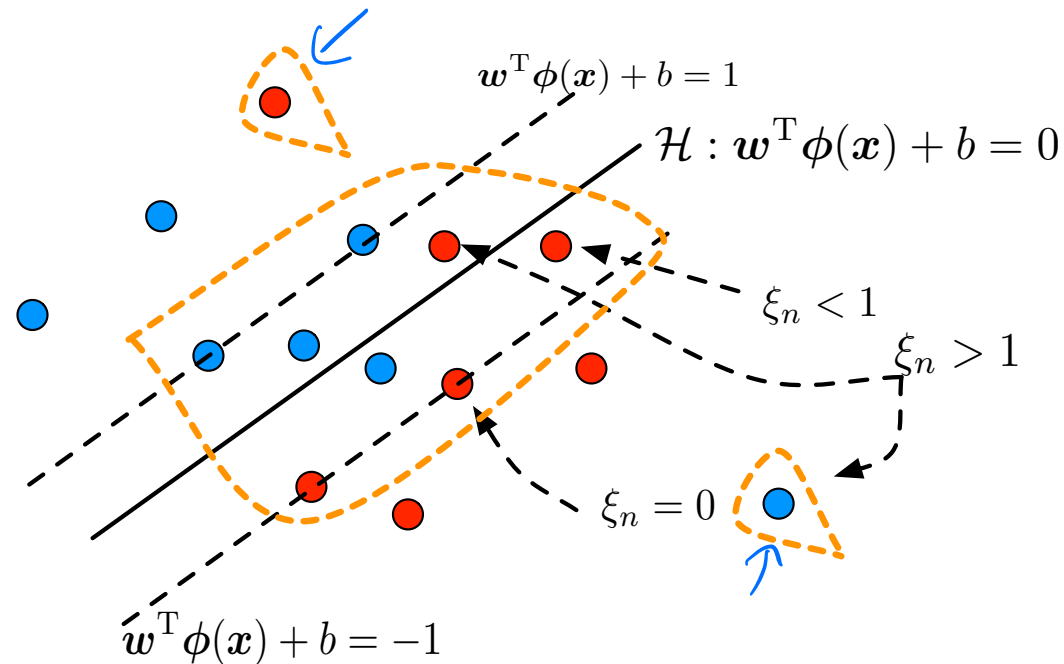
- This is a convex quadratic program: the objective function is quadratic in $\mathbf{w}$ and linear in ξ and the constraints are linear (inequality) constraints in $\mathbf{w}$, b and ξ_n .

How to Solve this Problem?

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

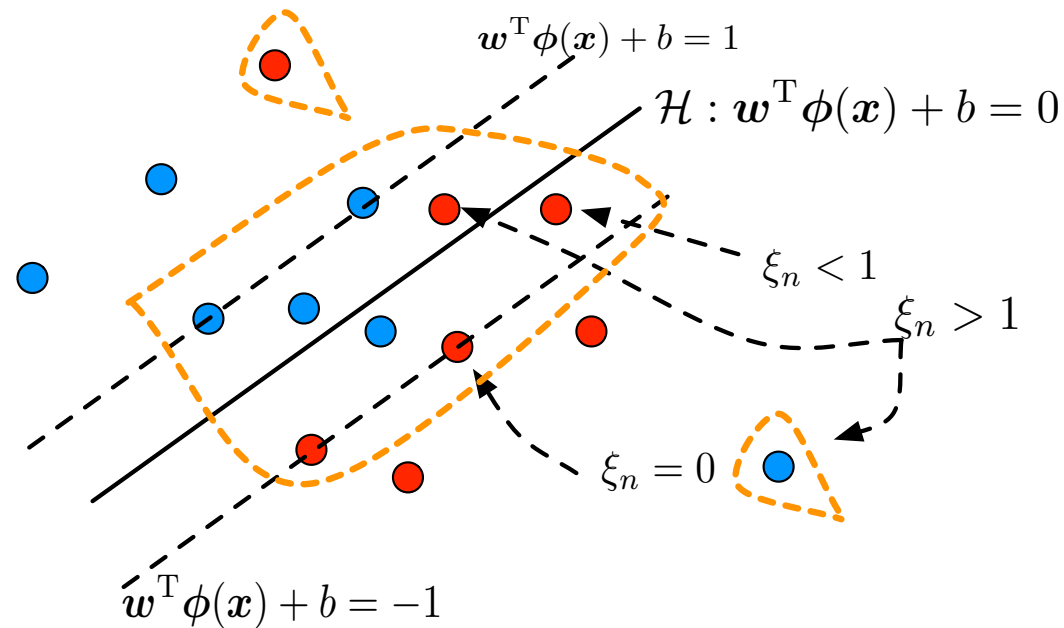
- This is a **convex quadratic program**: the objective function is quadratic in $\mathbf{w}$ and linear in ξ and the constraints are linear (inequality) constraints in $\mathbf{w}$, b and ξ_n .
- We can solve the optimization problem using general-purpose solvers, e.g., Matlab's `quadprog()` function, python's `scipy.optimize` package or CVXPY package.

Support Vectors: Revisit



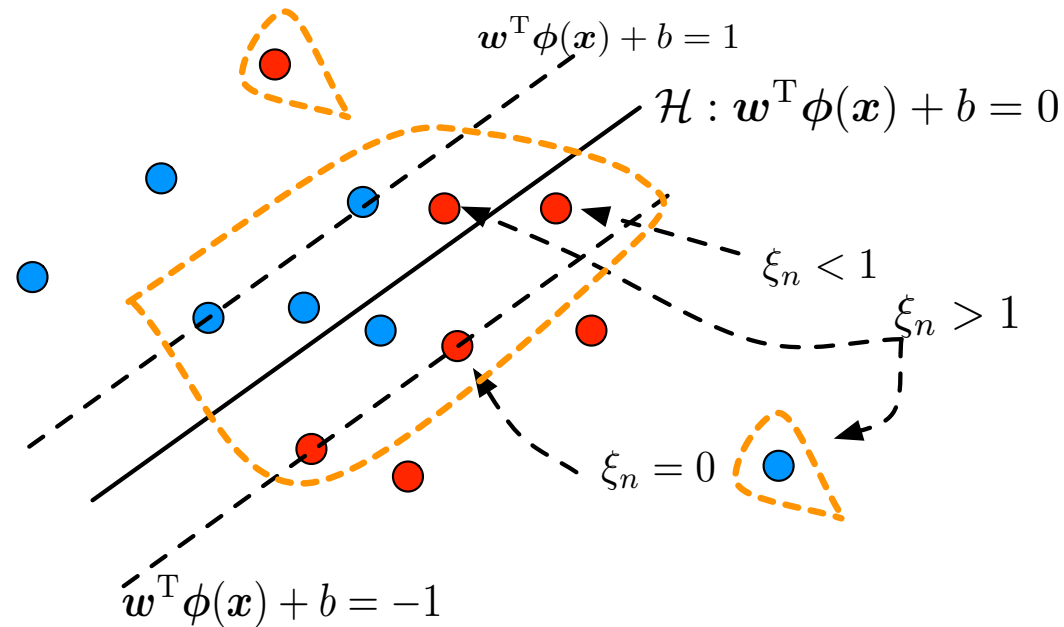
Support vectors are highlighted by the dotted orange lines. What does this mean mathematically?

Support Vectors: Revisit



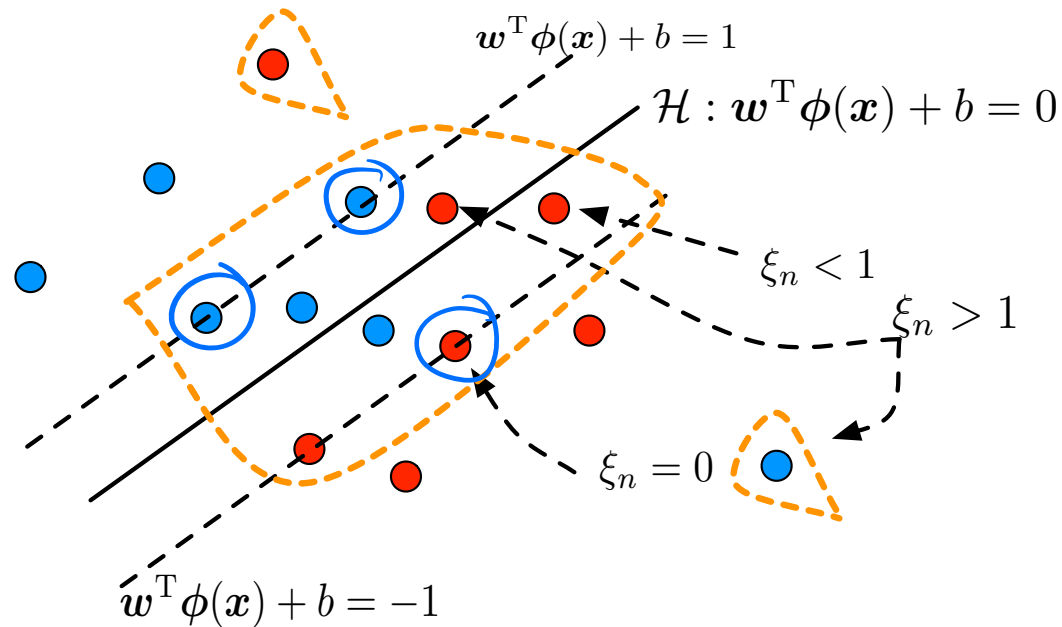
Recall the constraints $y_n[\mathbf{w}^T \mathbf{x}_n + b] \stackrel{=}{\geq} 1 - \xi_n$ from the soft-margin formulation. All the training points $(\mathbf{x}_n, y_n)$ that satisfy the constraint with “=” are support vectors.

Support Vectors: Revisit



In other words, support vectors satisfy $y_n[\mathbf{w}^T \mathbf{x}_n + b] = 1 - \xi_n$, which can be further divided into several categories:

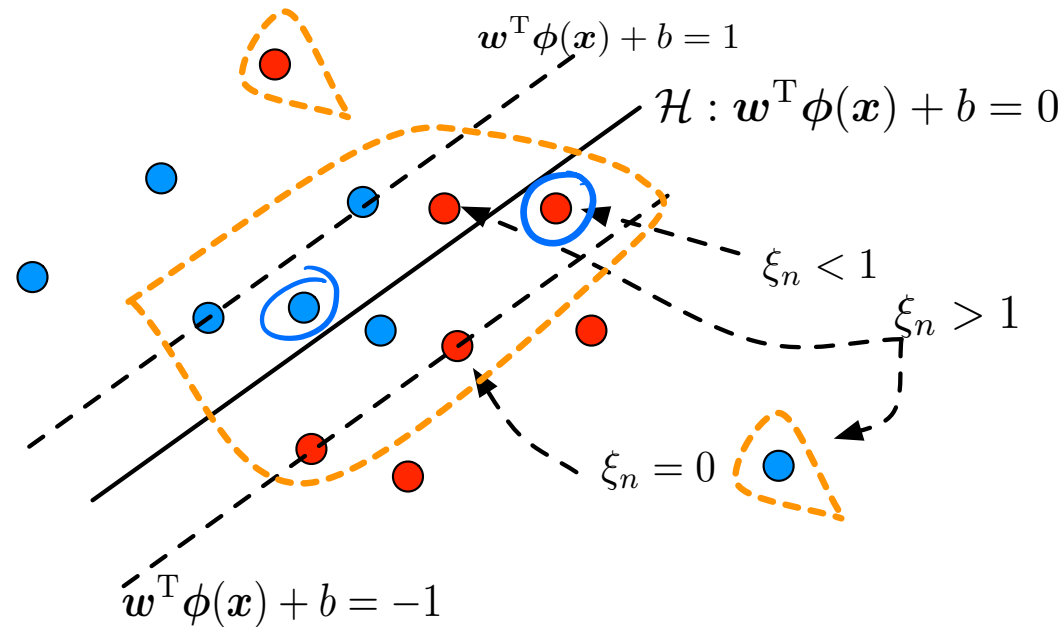
Support Vectors: Revisit



In other words, support vectors satisfy $y_n[\mathbf{w}^T \mathbf{x}_n + b] = 1 - \xi_n$, which can be further divided into several categories:

- $\xi_n = 0$: $y_n[\mathbf{w}^T \mathbf{x}_n + b] = 1$, the point is on the correct side with distance $\frac{1}{\|\mathbf{w}\|}$.

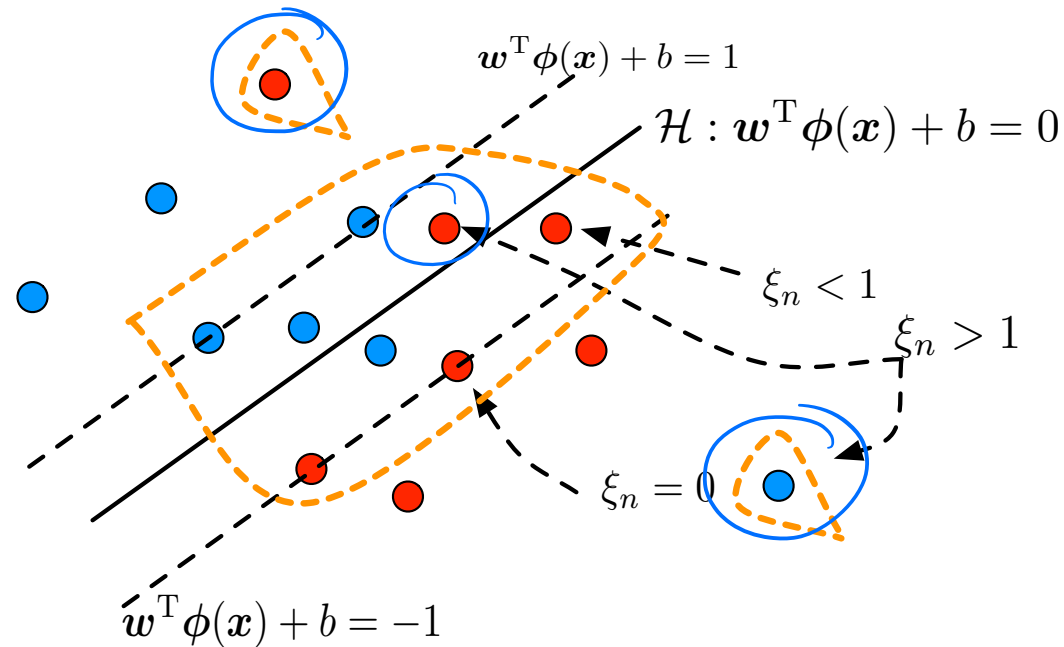
Support Vectors: Revisit



In other words, support vectors satisfy $y_n[\mathbf{w}^T \mathbf{x}_n + b] = 1 - \xi_n$, which can be further divided into several categories:

- $\xi_n = 0$: $y_n[\mathbf{w}^T \mathbf{x}_n + b] = 1$, the point is on the correct side with distance $\frac{1}{\|\mathbf{w}\|}$.
- $0 < \xi_n \leq 1$: $y_n[\mathbf{w}^T \mathbf{x}_n + b] \in [0, 1)$ on the correct side, but with distance less than $\frac{1}{\|\mathbf{w}\|}$.

Support Vectors: Revisit



In other words, support vectors satisfy $y_n[\mathbf{w}^T \mathbf{x}_n + b] = 1 - \xi_n$, which can be further divided into several categories:

- $\xi_n = 0$: $y_n[\mathbf{w}^T \mathbf{x}_n + b] = 1$, the point is on the correct side with distance $\frac{1}{\|\mathbf{w}\|}$.
- $0 < \xi_n \leq 1$: $y_n[\mathbf{w}^T \mathbf{x}_n + b] \in [0, 1)$ on the correct side, but with distance less than $\frac{1}{\|\mathbf{w}\|}$.
- $\xi_n > 1$: $y_n[\mathbf{w}^T \mathbf{x}_n + b] < 0$, on the wrong side of the boundary.

SVM: Hinge Loss Formulation

SVM vs. Logistic Regression

SVM soft-margin formulation

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

margin (pointing to $\frac{1}{2} \|\mathbf{w}\|_2^2$)
slacks (pointing to ξ_n)

Logistic regression formulation

$$\begin{aligned} \min_{\mathbf{w}} \quad & - \sum_n \{ y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) \\ & + (1 - y_n) \log [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)] \} \\ & + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \end{aligned}$$

$l(\sigma(\mathbf{w}^\top \mathbf{x}_n), y_n)$ (pointing to the log terms)
regularization (pointing to $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$)

- Logistic regression defines a loss for each data point and minimizes the total loss plus a regularization term.

SVM vs. Logistic Regression

SVM soft-margin formulation

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

Logistic regression formulation

$$\begin{aligned} \min_{\mathbf{w}} \quad & - \sum_n \{ y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) \\ & + (1 - y_n) \log [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)] \} \\ & + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \end{aligned}$$

- Logistic regression defines a **loss for each data point** and minimizes the total loss plus a regularization term.
- This is convenient for assessing the “goodness” of the model on each data point.

SVM vs. Logistic Regression

SVM soft-margin formulation

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

Logistic regression formulation

$$\begin{aligned} \min_{\mathbf{w}} \quad & - \sum_n \{ y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) \\ & + (1 - y_n) \log [1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)] \} \\ & + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \end{aligned}$$

- Logistic regression defines a **loss for each data point** and minimizes the total loss plus a regularization term.
- This is convenient for assessing the “goodness” of the model on each data point.
- Can we write SVMs in this form as well?

hinge loss

SVM vs. Logistic Regression

SVM soft-margin formulation

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \\ \text{s.t.} \quad & y_n [\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \forall n \\ & \xi_n \geq 0, \quad \forall n \end{aligned}$$

Logistic regression formulation

$$\begin{aligned} \min_{\mathbf{w}} \quad & - \sum_n \{y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) \\ & + (1 - y_n) \log[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\} \\ & + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \end{aligned}$$

- Logistic regression defines a **loss for each data point** and minimizes the total loss plus a regularization term.
- This is convenient for assessing the “goodness” of the model on each data point.
- Can we write SVMs in this form as well? **The Hinge Loss formulation!**

Derive the Hinge Loss Formulation

Here's the soft-margin formulation again:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \quad \text{s.t.} \quad y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \xi_n \geq 0, \quad \forall n$$

Derive the Hinge Loss Formulation

Here's the soft-margin formulation again:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \quad \text{s.t.} \quad y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \xi_n \geq 0, \quad \forall n$$

Now since $y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n \iff \xi_n \geq 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]$:

Derive the Hinge Loss Formulation

Here's the soft-margin formulation again:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \quad \text{s.t.} \quad y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \xi_n \geq 0, \quad \forall n$$

Now since $y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n \iff \xi_n \geq 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]$:

$$\min_{\mathbf{w}, b, \xi} C \sum_n \xi_n + \frac{1}{2} \|\mathbf{w}\|_2^2 \quad \text{s.t.} \quad \xi_n \geq \max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]), \quad \forall n$$

Derive the Hinge Loss Formulation

Here's the soft-margin formulation again:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \quad \text{s.t.} \quad y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \xi_n \geq 0, \quad \forall n$$

Now since $y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n \iff \xi_n \geq 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]$:

$$\min_{\mathbf{w}, b, \xi} C \sum_n \xi_n + \frac{1}{2} \|\mathbf{w}\|_2^2 \quad \text{s.t.} \quad \xi_n \geq \max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]), \quad \forall n$$

Now since the ξ_n should always be as small as possible, we obtain:

$$\min_{\mathbf{w}, b} C \sum_n \max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]) + \frac{1}{2} \|\mathbf{w}\|_2^2$$

Derive the Hinge Loss Formulation

Here's the soft-margin formulation again:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \quad \text{s.t.} \quad y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \quad \xi_n \geq 0, \quad \forall n$$

Now since $y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n \iff \xi_n \geq 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]$:

$$\min_{\mathbf{w}, b, \xi} C \sum_n \xi_n + \frac{1}{2} \|\mathbf{w}\|_2^2 \quad \text{s.t.} \quad \xi_n \geq \max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]), \quad \forall n$$

Now since the ξ_n should always be as small as possible, we obtain:

$$\min_{\mathbf{w}, b} C \sum_n \max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]) + \frac{1}{2} \|\mathbf{w}\|_2^2$$

Divide by C and set $\lambda = \frac{1}{C}$, we get the **Hinge Loss formulation**:

$$\min_{\mathbf{w}, b} \sum_n \underbrace{\max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b])}_{\text{Hinge Loss for } x_n, y_n} + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

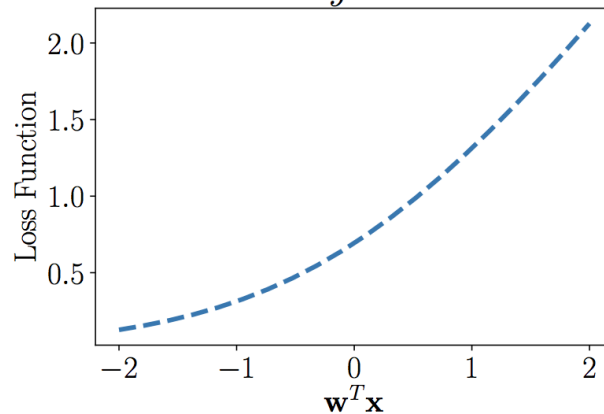
Cross-Entropy Loss vs. Hinge Loss

Given training data $(\mathbf{x}_n, y_n)$, the cross entropy loss was

$$-\{y_n \log \sigma(\mathbf{w}^\top \mathbf{x}_n) + (1 - y_n) \log[1 - \sigma(\mathbf{w}^\top \mathbf{x}_n)]\}$$

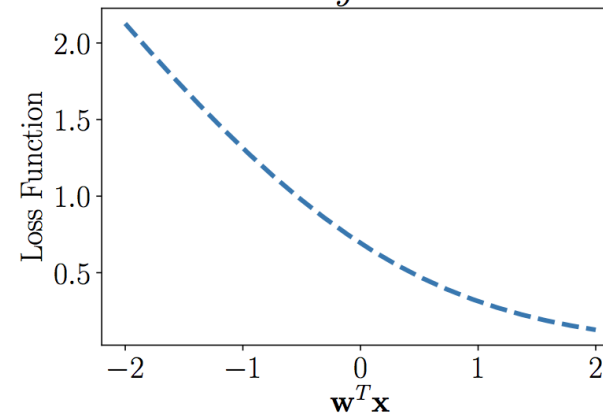
$$-\log \left(\frac{e^{-\mathbf{w}^\top \mathbf{x}_n}}{1 + e^{-\mathbf{w}^\top \mathbf{x}_n}} \right)$$

If $y = 0$



$$-\log \left(\frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}_n}} \right)$$

If $y = 1$



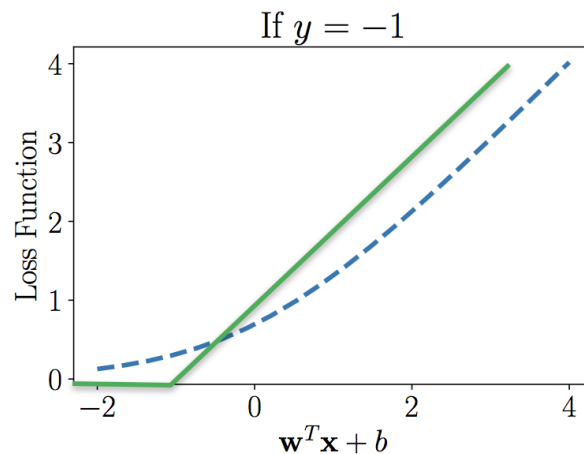
What does the Hinge Loss Function look like?

Cross-Entropy Loss vs. Hinge Loss

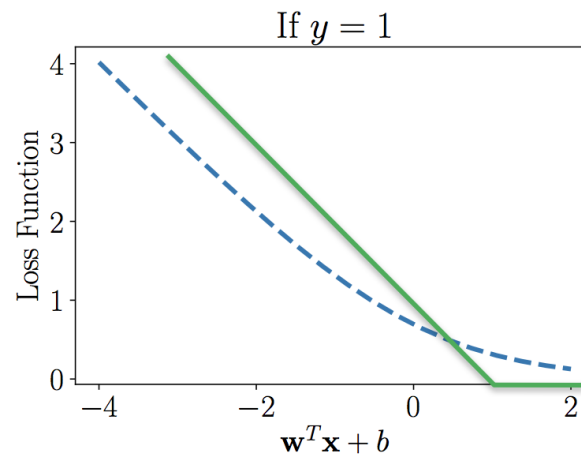
Given training data $(\mathbf{x}_n, y_n)$, the Hinge loss is

$$\max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b])$$

$$\max(\mathbf{w}^\top \mathbf{x}_n + b + 1, 0)$$



$$\max(1 - \mathbf{w}^\top \mathbf{x}_n - b, 0)$$



- Loss grows linearly as we move away from the boundary.
- No penalty if a point is more than 1 unit from the boundary.
- Makes the search for the boundary easier (as we will see later).

Hinge Loss SVM Formulation

Minimizing the total hinge loss on all the training data

$$\min_{\mathbf{w}, b} \sum_n \underbrace{\max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b])}_{\text{hinge loss for sample } n} + \underbrace{\frac{\lambda}{2} \|\mathbf{w}\|_2^2}_{\text{regularizer}}$$

Analogous to regularized least squares or logistic regression, as we balance between two terms (the loss and the regularizer).

Hinge Loss SVM Formulation

Minimizing the total hinge loss on all the training data

$$\min_{\mathbf{w}, b} \sum_n \underbrace{\max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b])}_{\text{hinge loss for sample } n} + \underbrace{\frac{\lambda}{2} \|\mathbf{w}\|_2^2}_{\text{regularizer}}$$

Analogous to regularized least squares or logistic regression, as we balance between two terms (the loss and the regularizer).

- Can solve using gradient descent to get the optimal $\mathbf{w}$ and b

Hinge Loss SVM Formulation

Minimizing the total hinge loss on all the training data

$$\min_{\mathbf{w}, b} \sum_n \underbrace{\max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b])}_{\text{hinge loss for sample } n} + \underbrace{\frac{\lambda}{2} \|\mathbf{w}\|_2^2}_{\text{regularizer}}$$

Analogous to regularized least squares or logistic regression, as we balance between two terms (the loss and the regularizer).

- Can solve using gradient descent to get the optimal $\mathbf{w}$ and b
- Gradient of the first term will be either 0, $\mathbf{x}_n$ or $-\mathbf{x}_n$ depending on y_n and $\mathbf{w}^\top \mathbf{x}_n + b$.

Hinge Loss SVM Formulation

Minimizing the total hinge loss on all the training data

$$\min_{\mathbf{w}, b} \sum_n \underbrace{\max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b])}_{\text{hinge loss for sample } n} + \underbrace{\frac{\lambda}{2} \|\mathbf{w}\|_2^2}_{\text{regularizer}}$$

Analogous to regularized least squares or logistic regression, as we balance between two terms (the loss and the regularizer).

- Can solve using gradient descent to get the optimal $\mathbf{w}$ and b
- Gradient of the first term will be either 0, $\mathbf{x}_n$ or $-\mathbf{x}_n$ depending on y_n and $\mathbf{w}^\top \mathbf{x}_n + b$.
- Much easier to compute than in logistic regression, where we need to compute the sigmoid function $\sigma(\mathbf{w}^\top \mathbf{x}_n + b)$ in each iteration.

Three SVM Formulations

Hard-margin (for separable data)

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 \text{ s.t. } y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1, \xi_n \geq 0, \forall n$$

Soft-margin (add slack variables)

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_n \xi_n \text{ s.t. } y_n[\mathbf{w}^\top \mathbf{x}_n + b] \geq 1 - \xi_n, \xi_n \geq 0, \forall n$$

Hinge loss (define a loss function for each data point)

$$\min_{\mathbf{w}, b} \sum_n \max(0, 1 - y_n[\mathbf{w}^\top \mathbf{x}_n + b]) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

You should know:

- Max-margin formulation for separable and non-separable SVMs.
- Definition and importance of support vectors.
- Hinge loss formulation of SVMs.
- Equivalence of the max-margin and hinge loss formulations.